Anjuman-I-Islam's

AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS

M. S. Ali Road, Do Taaki, Grant Road (E), Mumbai-8.

PRACTICAL JOURNAL

SUBMITTED TO

T.Y. B.Sc. (Data Science)

Semester - VI

Applied Business Analytics

University of Mumbai

By

Under the guidance of

Prof. Hanna Sadiya Shaikh

DEPARTMENT OF DATA SCIENCE

AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS

Academic Year (2025 – 2026)

Anjuman-I-Islam's

"AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS"

M.S. Ali Road, Mumbai - 400008

CERTIFICATE

This is a certify that of The class **T.Y.B.Sc - Data Science** Roll No: University Exam Seat No: _____has satisfactorily completed the Practical Journal in the subject of **Applied Business Analytics** as Prescribed in the syllabus of **Third Year Bachelor of Science in Data Science** During the academic year 2025 – 2026

B.Sc. Data Science Coordinator (Signature)	**External Examiner (Signature)**	**Lecturer-in-Charge (Signature)**

Index

Practical No.		Name of Experiment	Date	Signature
Q1.	**Introduction to Business Analytics**		**6/11/2025**	
	(A)	Collect data from a real-life business scenario and perform exploratory data analysis (EDA) to gain insights into the dataset	6/11/2025	
	(B)	Analyze customer data to identify trends and patterns that can be used for business decision-making.	6/11/2025	
Q2.	**Describing the Distribution of a Variable:**		**13/11/2025**	
	(A)	Obtain a dataset and calculate descriptive statistics (mean, median, mode, variance, etc.) for a specific variable of interest.	13/11/2025	
	(B)	Create visualizations (histograms, box plots) to depict the distribution of a variable and analyze its characteristics.	13/11/2025	
Q3.	**Finding Relationships Among Variables:**		**20/11/2025**	
	(A)	Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.	20/11/2025	
	(B)	Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).	20/11/2025	
Q4.	**Business Intelligence Tools for Data Analysis:**		**27/11/2025**	
	(A)	Utilize a business intelligence tool (e.g., Tableau, Power BI) to extract insights from a dataset and create interactive visualizations for effective data analysis	27/11/2025	
Q5.	**Probability and Probability Distributions:**		**11/12/2025**	
	(A)	Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.	18/12/2025	
	(B)	Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.	18/12/2025	
Q6.	**Decision Making under Uncertainty:**		**8/01/2026**	
	(A)	Develop a decision tree model to make business decisions considering uncertainties and associated probabilities at each decision point.	8/01/2026	

	(B)	Apply the concept of expected value to evaluate different decision alternatives and select the optimal one.	8/01/2025	
Q7.		**Sampling and Sampling Distributions:**	**15/01/2026**	
	(A)	Conduct a survey and collect data from a sample population, ensuring proper sampling techniques are employed.	15/01/2026	
	(B)	Use the Central Limit Theorem to analyze the sampling distribution of a sample mean and estimate population parameters.	15/01/2026	
Q8.		**Confidence Interval Estimation:**	**22/01/2026**	
	(A)	Calculate confidence intervals for population means or proportions using sample data and interpret the results in a business context.	22/01/2026	
	(B)	Apply bootstrapping techniques to estimate confidence intervals for non-parametric statistics.	22/01/2026	
Q9.		**Hypothesis Testing:**	**29/01/2026**	
	(A)	Formulate null and alternative hypotheses related to a business problem, conduct a hypothesis test using appropriate statistical tests, and interpret the results.	29/01/2026	
	(B)	Perform A/B testing on a website or marketing campaign to evaluate the effectiveness of different strategies and make data-driven decisions.	29/01/2026	
Q10.		**Regression Analysis and Time Series Analysis:**	**5/02/2026**	
	(A)	Develop a regression model to predict future sales based on historical data, assess model performance, and interpret the significance of predictor variables.	5/02/2026	
	(B)	Apply time series analysis techniques (e.g., ARIMA, exponential smoothing) to forecast future demand for a product or service, and evaluate the accuracy of the forecasts.	26/02/2026	

Applied Business Analytics # Practical No: 1

Name		**Roll No:**	
Class	T.Y.B.Sc. Data Science	**Seat Number:**	
Subject / Topic	Applied Business Analytics		
Topic	Introduction to Business Analytics		

Practical No: 1 Introduction to Business Analytics

1(A) Collect data from a real-life business scenario and perform exploratory data analysis (EDA) to gain insights into the dataset.

Code:

```
# Import necessary libraries
import pandas as pd import
matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
# Ensure the path is properly enclosed in quotes and formatted file_name =
"D:/Sem 6 Practicals/ABA Practicals/Global_Superstore(CSV).csv" df =
pd.read_csv(file_name) # Display basic dataset information print("Dataset
Shape:", df.shape) print("\nDataset Info:")
df.info() print("\nMissing Values:\n",
df.isnull().sum())
# Display summary statistics print("\nSummary
Statistics (Numerical Data):") print(df.describe())
print("\nSummary Statistics (Categorical
Data):") print(df.describe(include=['object']))
```

```
# Handle missing data (optional, adjust as needed) df =
df.dropna(subset=['Postal Code'])  # Drop rows with missing Postal Code
# Univariate Analysis: Visualize sales
distribution sns.histplot(df['Sales'], kde=True)
plt.title("Sales Distribution") plt.xlabel("Sales")
plt.ylabel("Frequency") plt.show()
# Bivariate Analysis: Sales by Category
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Category',
y='Sales') plt.title("Sales by Category")
plt.show()
# Correlation Analysis: Heatmap
# Ensure all columns are present in the correlation matrix
correlation_columns = ['Sales', 'Profit', 'Discount',
'Quantity'] if 'Shipping Cost' in df.columns:  # Check if the
column exists     correlation_columns.append('Shipping
Cost') correlation = df[correlation_columns].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix") plt.show() # Derive Insights print("\nTop 5 Products by
Sales:") print(df.groupby('Product
Name')['Sales'].sum().sort_values(ascending=False).head()) print("\nSales by
Region:") print(df.groupby('Region')['Sales'].sum())
# Monthly Sales Trend
if 'Order Date' in df.columns:
    df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Month'] = df['Order Date'].dt.to_period('M')
monthly_sales = df.groupby('Month')['Sales'].sum()
```

```
    plt.figure(figsize=(12, 6))
monthly_sales.plot(kind='line', marker='o')
plt.title("Monthly Sales Trend")
plt.xlabel("Month")      plt.ylabel("Total
Sales")
    plt.grid()
plt.show() else:
    print("No 'Order Date' column found for monthly trend analysis.")
```

Output:

```
Dataset Shape: (1000, 24)
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 24 columns):
 #   Column         Non-Null Count  Dtype
---  -------        ----------------  ------
0    Row ID         1000 non-null   int64
1    Order ID       1000 non-null   object
2    Order Date     1000 non-null   object
3    Ship Date      1000 non-null   object
4    Ship Mode      1000 non-null   object
5    Customer ID    1000 non-null   object
6    Customer Name  1000 non-null   object
7    Segment        1000 non-null   object
8    Postal Code    194 non-null    float64
9    City           1000 non-null   object
10   State          1000 non-null   object
11   Country        1000 non-null   object
12   Region         1000 non-null   object
```

13 Market 1000 non-null object

14 Product ID 1000 non-null object

15 Category 1000 non-null object

16 Sub-Category 1000 non-null object

17 Product Name 1000 non-null object

18 Sales 1000 non-null float64

19 Quantity 1000 non-null int64

20 Discount 1000 non-null float64

21 Profit 1000 non-null float64

22 Shipping Cost 1000 non-null float64 23 Order Priority 1000 non-null object

dtypes: float64(5), int64(2), object(17)

memory usage: 187.6+ KB

Missing Values:

Row ID 0

Order ID 0

Order Date 0

Ship Date 0

Ship Mode 0

Customer ID 0

Customer Name 0

Segment 0

Postal Code 806

City 0

State 0

Country 0

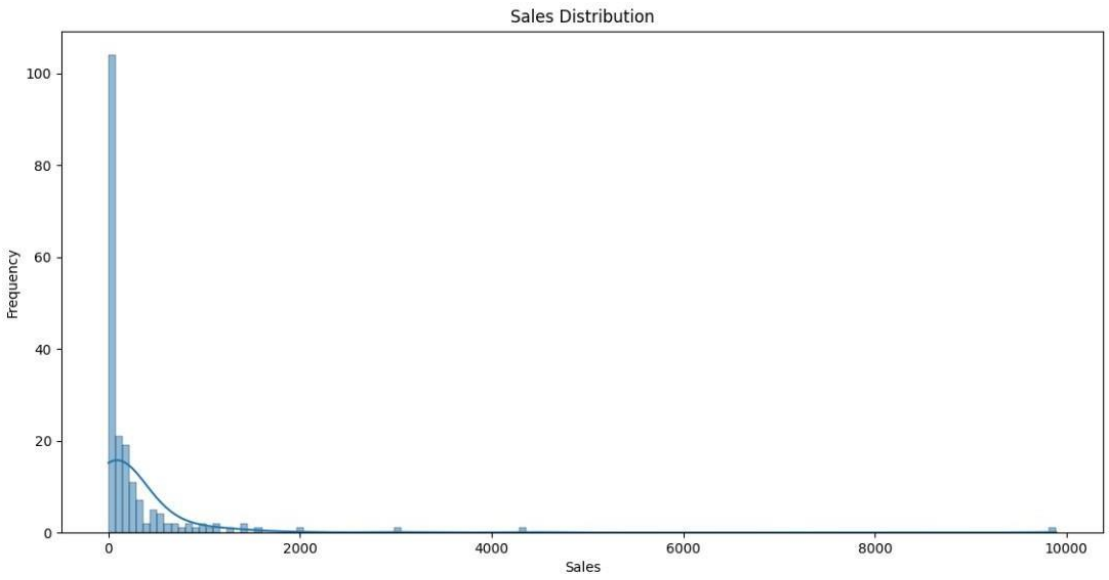
Region 0

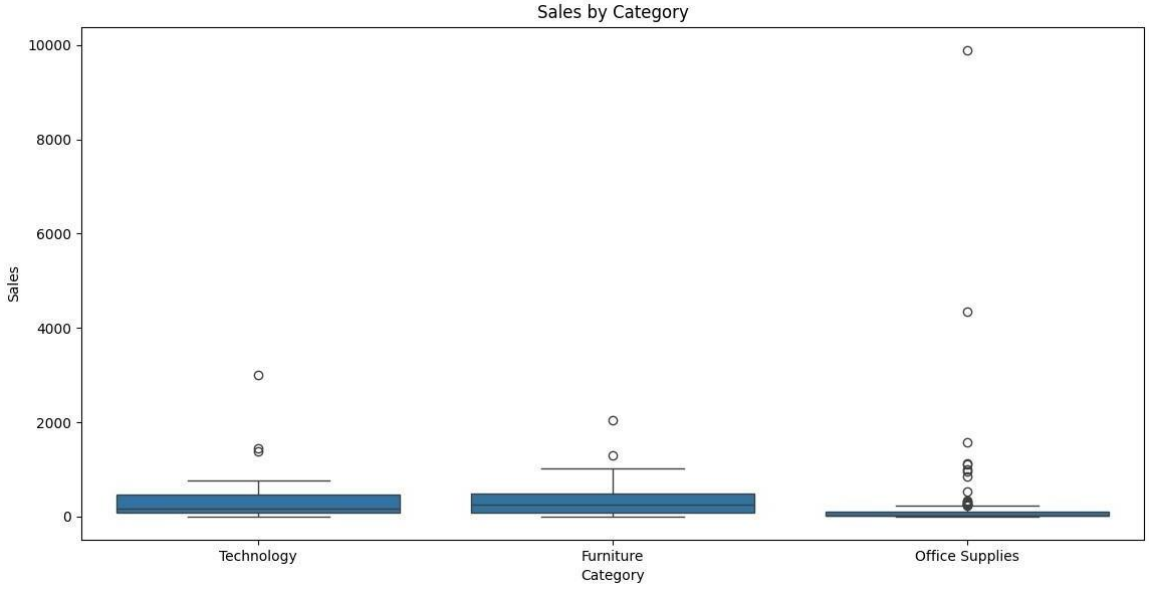
Market 0

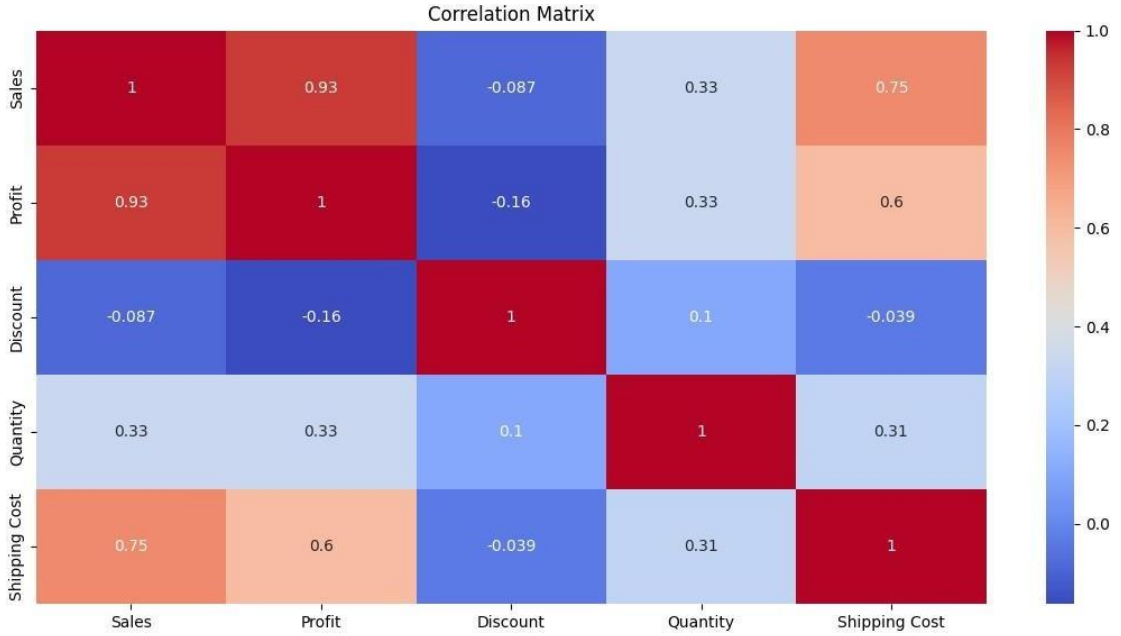
Product ID 0

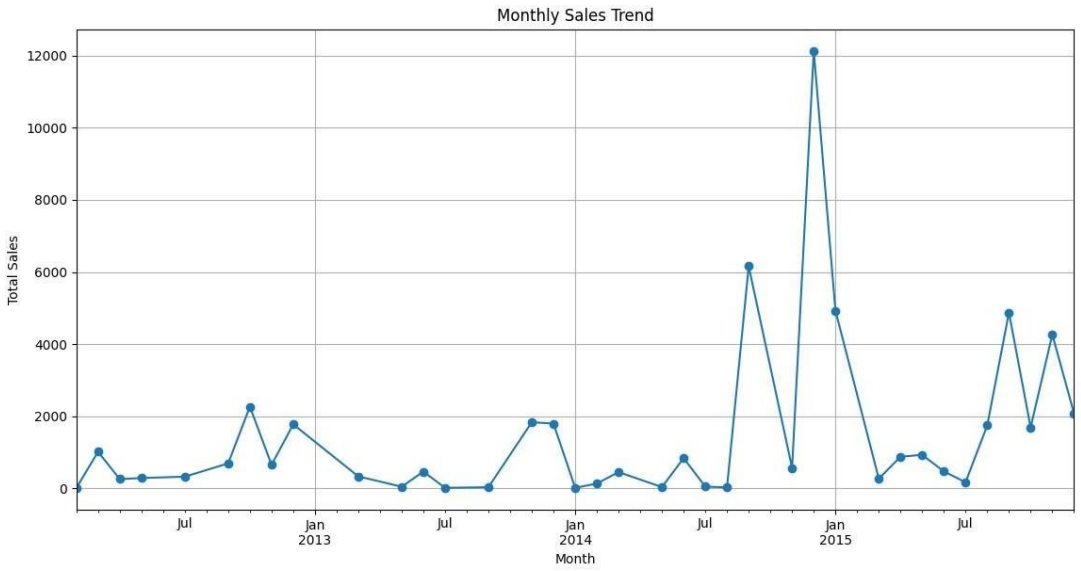
Category 0

```
Sub-Category       0
Product Name       0
Sales              0
Quantity           0
Discount           0
Profit             0
Shipping Cost      0
Order Priority     0
dtype: int64
Summary Statistics (Numerical Data):
            Row ID   Postal Code  ...       Profit  Shipping Cost
count   1000.000000    194.000000  ...  1000.000000    1000.000000
mean   25079.328000  53966.170103  ...   288.920440     272.384897
std    12897.726632  33734.306466  ...   574.504782     176.160716
min       58.000000   2920.000000  ... -3059.820000       1.070000
25%    15118.750000  19134.000000  ...    10.037500     209.827500
50%    25084.500000  60564.000000  ...   190.685000     258.897500
75%    34524.000000  88187.500000  ...   518.872500     351.070250
max    51284.000000  98198.000000  ...  4946.370000     923.630000
[8 rows x 7 columns]
Summary Statistics (Categorical Data):
                    Order ID  ... Order Priority count
1000  ...          1000 unique                    867
...            4
top     CA-2014-AH10120140-41992  ...          High
freq                          7  ...           372
[4 rows x 17 columns]
```


Sales Distribution
Frequency
Sales
0
20
40
60
80
100
0
2000
4000
6000
8000
10000


Sales by Category
Sales
Category
Technology
Furniture
Office Supplies
0
2000
4000
6000
8000
10000


Correlation Matrix
Sales
Profit
Discount
Quantity
Shipping Cost
1
0.93
-0.087
0.33
0.75
0.93
1
-0.16
0.33
0.6
-0.087
-0.16
1
0.1
-0.039
0.33
0.33
0.1
1
0.31
0.75
0.6
-0.039
0.31
1
1.0
0.8
0.6
0.4
0.2
0.0


Monthly Sales Trend
Total Sales
12000
10000
8000
6000
4000
2000
0
Jul
Jan
2013
Jul
Jan
2014
Jul
Jan
2015
Jul
Month

1(B) Analyze customer data to identify trends and patterns that can be used for business decision-making.

Code:

```
# Import necessary libraries import pandas as pd

import matplotlib.pyplot as plt import seaborn as

sns from sklearn.cluster import KMeans from

sklearn.preprocessing import StandardScaler

# Load the dataset

file_name = "D:/Sem 6 Practicals/ABA Practicals/customer_data.csv"  # Replace with your
file path

df = pd.read_csv(file_name) #

Display basic dataset information

print("Dataset Shape:", df.shape)

print("\nDataset Info:")

df.info() print("\nMissing Values:\n",

df.isnull().sum())

# Display summary statistics print("\nSummary

Statistics (Numerical Data):") print(df.describe())

print("\nSummary Statistics (Categorical

Data):") print(df.describe(include=['object']))

# Visualize demographics

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x='Gender')

plt.title("Gender Distribution")

plt.show()  # Show the plot

plt.figure(figsize=(10, 6))

sns.histplot(df['Age'], kde=True, bins=20)

plt.title("Age Distribution")
```

```
plt.xlabel("Age") plt.ylabel("Frequency")
plt.show()  # Show the plot
# Sales Trends: Analyzing purchase trends over time if
'Purchase Date' in df.columns:
    df['Purchase Date'] = pd.to_datetime(df['Purchase Date'])
df['Month'] = df['Purchase Date'].dt.to_period('M')
monthly_sales = df.groupby('Month')['Sales'].sum()
    plt.figure(figsize=(12, 6))
monthly_sales.plot(kind='line', marker='o')
plt.title("Monthly Sales Trend")
plt.xlabel("Month")      plt.ylabel("Total
Sales")
    plt.grid()
    plt.show()  # Show the plot
# RFM Analysis if 'Customer ID' in df.columns and
'Sales' in df.columns:
    rfm = df.groupby('Customer ID').agg({
        'Purchase Date': lambda x: (pd.Timestamp.today() - x.max()).days,  # Recency
        'Customer ID': 'count',  # Frequency
        'Sales': 'sum'  # Monetary
    })
    rfm.columns = ['Recency', 'Frequency',
'Monetary']      print("\nRFM Head:")
print(rfm.head())
    # Segment customers based on RFM scores
    rfm['RFM Score'] = (rfm['Recency'].rank(ascending=False) +
                        rfm['Frequency'].rank() +
rfm['Monetary'].rank())
```

```
    top_customers = rfm.sort_values('RFM Score', ascending=False).head()
print("\nTop Customers by RFM Score:")    print(top_customers)
# Customer Segmentation using K-Means
if 'Age' in df.columns and 'Annual Income (k$)' in df.columns:
    clustering_data = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
clustering_data = StandardScaler().fit_transform(clustering_data)    kmeans
= KMeans(n_clusters=3, random_state=42)    df['Customer Segment'] =
kmeans.fit_predict(clustering_data)
    # Visualize the segments
plt.figure(figsize=(10, 6))
    sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
hue='Customer Segment', palette='viridis')    plt.title("Customer
Segmentation")    plt.show()  # Show the plot
```

Output:

```
Dataset Shape: (40, 7)
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 7 columns):
 #   Column                  Non-Null Count  Dtype
--  -------                 ----------------  ------
0   Customer ID             40 non-null     int64
1   Gender                  40 non-null     object
2   Age                     40 non-null     int64
3   Annual Income (k$)      40 non-null     int64
4   Spending Score (1-100)  40 non-null     int64
5   Purchase Date           40 non-null     object
6   Sales                   40 non-null     int64
dtypes: int64(5), object(2)
```

memory usage: 2.3+ KB Missing

Values:

Customer ID 0

Gender 0

Age 0

Annual Income (k$) 0

Spending Score (1-100) 0

Purchase Date 0

Sales 0

dtype: int64

Summary Statistics (Numerical Data):

Customer ID Age ... Spending Score (1-100) Sales count

40.000000 40.000000 … 40.000000 40.000000 mean

20.500000 28.175000 … 67.700000 443.000000 std

11.690452 5.329586 … 20.536646 152.890106 min

1.000000 19.000000 … 6.000000 150.000000

25% 10.750000 24.000000 ... 54.250000 320.000000

50% 20.500000 27.500000 ... 71.000000 450.000000

75% 30.250000 31.250000 … 84.250000 532.500000

max 40.000000 40.000000 … 95.000000 740.000000

[8 rows x 5 columns]

Summary Statistics (Categorical Data):

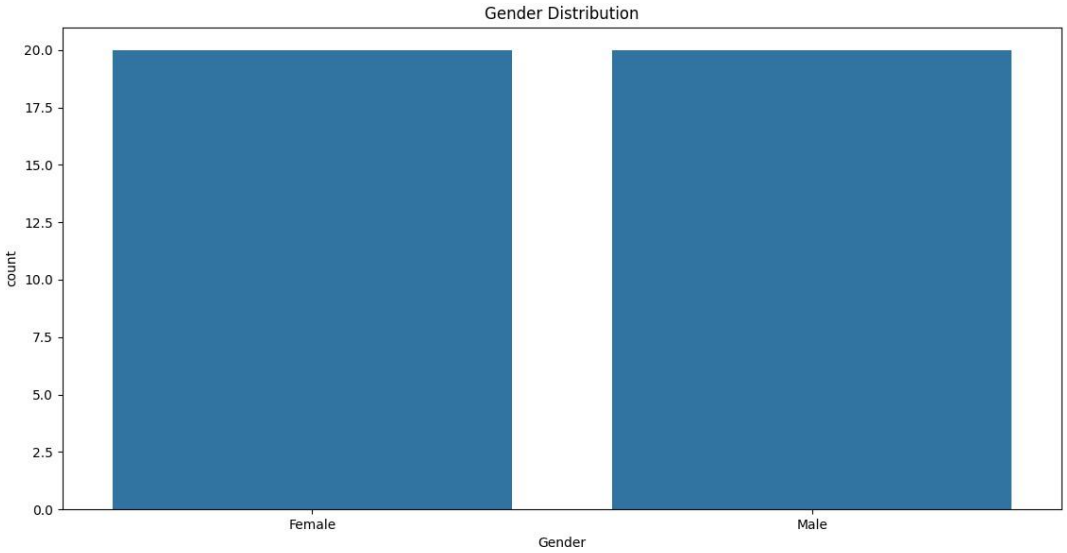
Gender Purchase Date

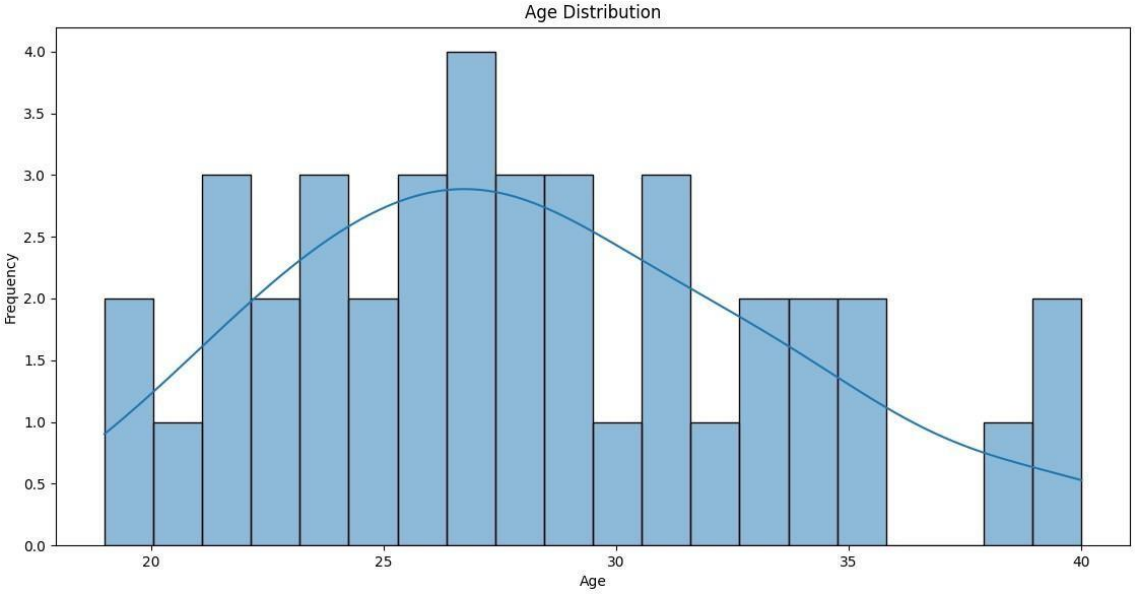
count 40 40

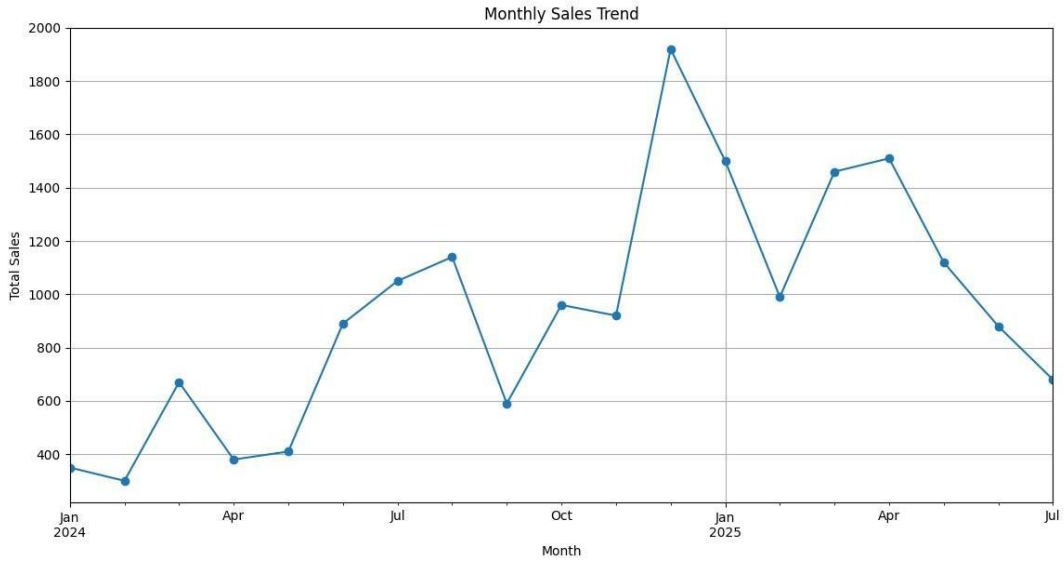
unique 2 40

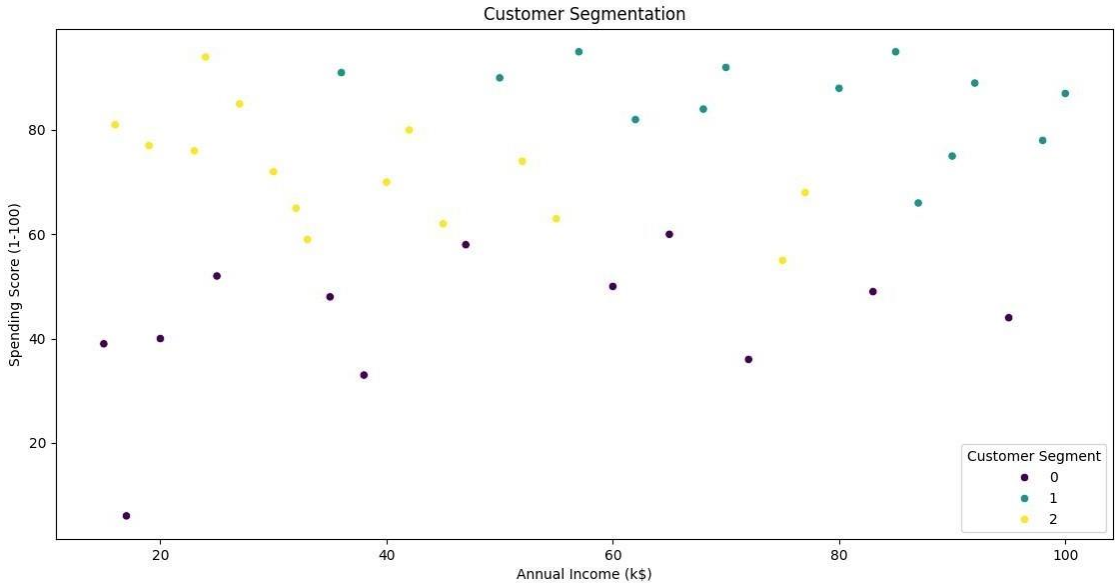
top Female 2024-01-15

freq 20 1


Gender Distribution
count
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
Female
Male
Gender


Age Distribution
Frequency
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
20
25
30
35
40
Age


Monthly Sales Trend
Total Sales
2000
1800
1600
1400
1200
1000
800
600
400
Jan
2024
Apr
Jul
Oct
Jan
2025
Apr
Jul
Month


Customer Segmentation
Spending Score (1-100)
80
60
40
20
20
40
60
80
100
Annual Income (k$)
Customer Segment
0
1
2

Applied Business Analytics # Practical No: 2

Name		**Roll No:**	
Class	T.Y.B.Sc. Data Science	**Seat Number:**	
Subject / Topic	Applied Business Analytics		
Topic	Describing the Distribution of a Variable		

Practical No: 2 Describing the Distribution of a Variable

2(A) Obtain a dataset and calculate descriptive statistics (mean, median, mode, variance, etc.) for a specific variable of interest.

Code:

```
import pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns import warnings
warnings.filterwarnings('ignore'
)
# Load the dataset
file_name = "D:/Sem 6 Practicals/ABA Practicals/fortune500.csv"  # Replace with your file
path df = pd.read_csv(file_name) # Analyze the column 'lon' (longitude) column = 'lon' #
Descriptive Statistics
mean = df[column].mean() median
= df[column].median() mode =
df[column].mode()[0] minimum =
df[column].min() maximum =
df[column].max() range_ =
maximum - minimum variance =
```

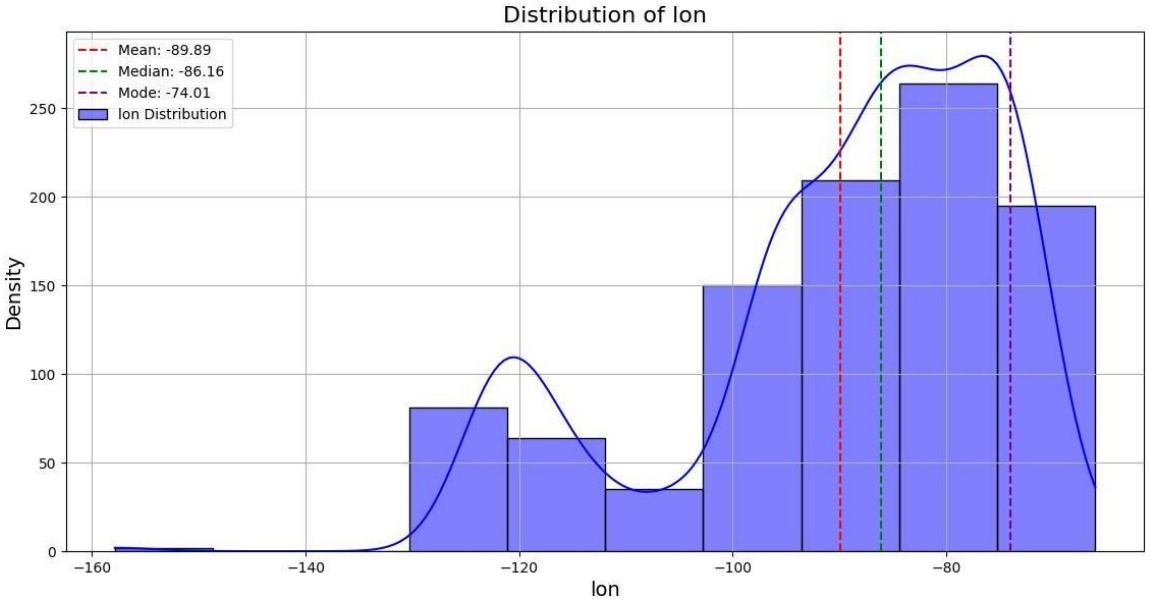
```
df[column].var() std_dev =
df[column].std() q1 =
df[column].quantile(0.25) q2 =
df[column].quantile(0.5)  #
Median q3 =
df[column].quantile(0.75) iqr = q3
- q1
skewness = df[column].skew() kurtosis
= df[column].kurt()

# Combine statistics into a DataFrame for better presentation statistics
= pd.DataFrame({
    'Statistic': ['Mean', 'Median', 'Mode', 'Minimum', 'Maximum', 'Range',
                  'Variance', 'Standard Deviation', 'Q1 (25%)', 'Q2 (Median)',
                  'Q3 (75%)', 'IQR', 'Skewness', 'Kurtosis'],
    'Value': [mean, median, mode, minimum, maximum, range_, variance, std_dev,
              q1, q2, q3, iqr, skewness, kurtosis]
})
# Print Descriptive Statistics print("\nDescriptive Statistics:") print(statistics) # Plot
Distribution plt.figure(figsize=(10, 6)) sns.histplot(df[column], bins=10, kde=True,
color='blue', label=f'{column} Distribution') plt.axvline(mean, color='red', linestyle='--',
label=f'Mean: {mean:.2f}') plt.axvline(median, color='green', linestyle='--',
label=f'Median: {median:.2f}') plt.axvline(mode, color='purple', linestyle='--',
label=f'Mode: {mode:.2f}') plt.title(f"Distribution of {column}", fontsize=16)
plt.xlabel(column, fontsize=14) plt.ylabel("Density", fontsize=14)

plt.legend() plt.grid()
plt.show()
# Boxplot
```

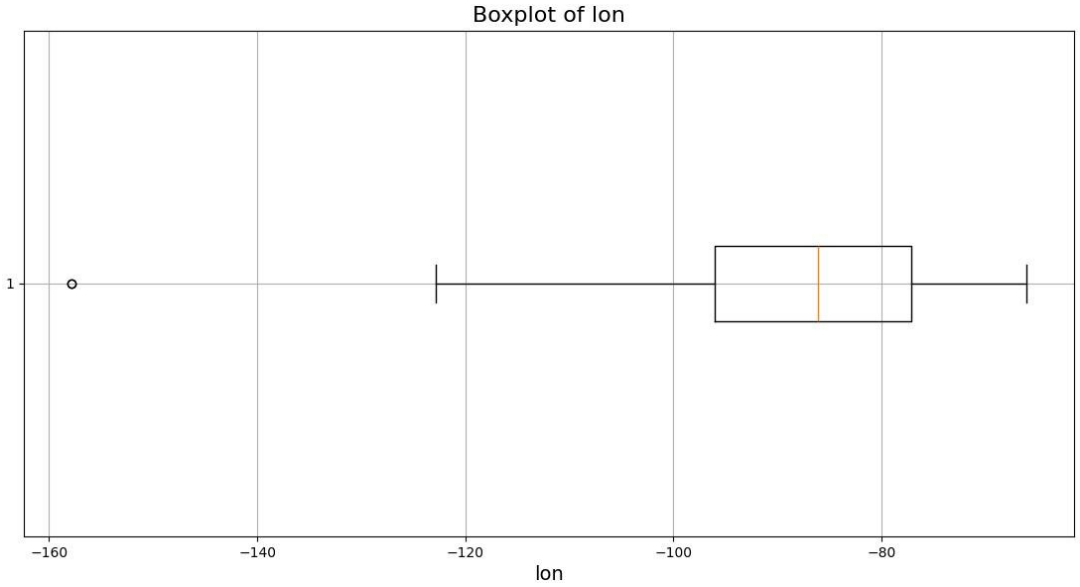
```
plt.figure(figsize=(8, 5))
plt.boxplot(df[column], vert=False)
plt.title(f"Boxplot of {column}", fontsize=16)
plt.xlabel(column, fontsize=14)
plt.grid()
plt.show()
```

Output:

Descriptive Statistics:

	Statistic	Value
0	Mean	-89.886327
1	Median	-86.158100
2	Mode	-74.005900
3	Minimum	-157.858000
4	Maximum	-66.060200
5	Range	91.797800
6	Variance	237.597490
7	Standard Deviation	15.414198
8	Q1 (25%)	-95.994100
9	Q2 (Median)	-86.158100
10	Q3 (75%)	-77.177300
11	IQR	18.816800
12	Skewness	-1.008653
13	Kurtosis	0.342740


Distribution of lon
Mean: -89.89
Median: -86.16
Mode: -74.01
lon Distribution
Density
lon
0
50
100
150
200
250
−160
−140
−120
−100
−80


Boxplot of lon
1
lon
−160
−140
−120
−100
−80

2(B) Create visualizations (histograms, box plots) to depict the distribution of a variable and analyze its characteristics.

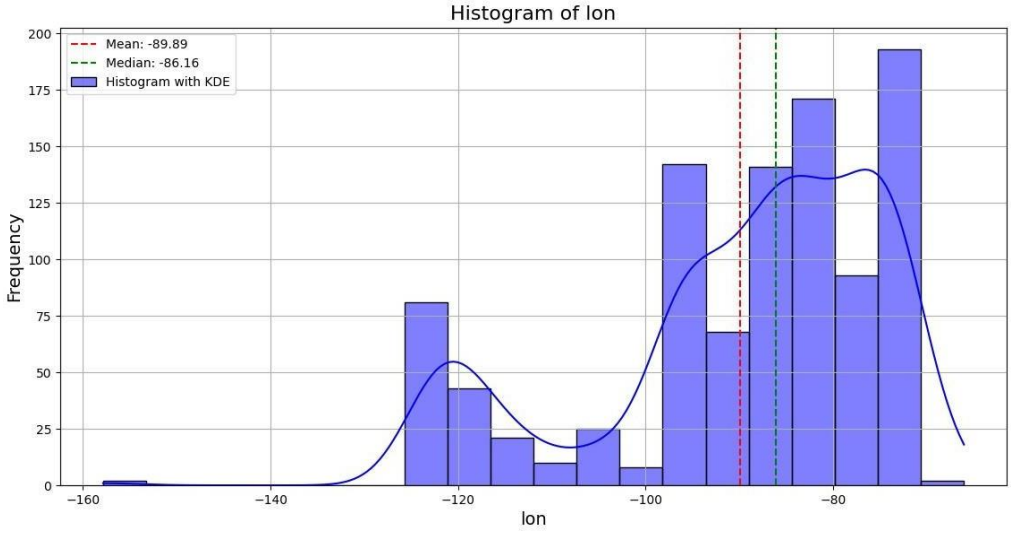
Code:

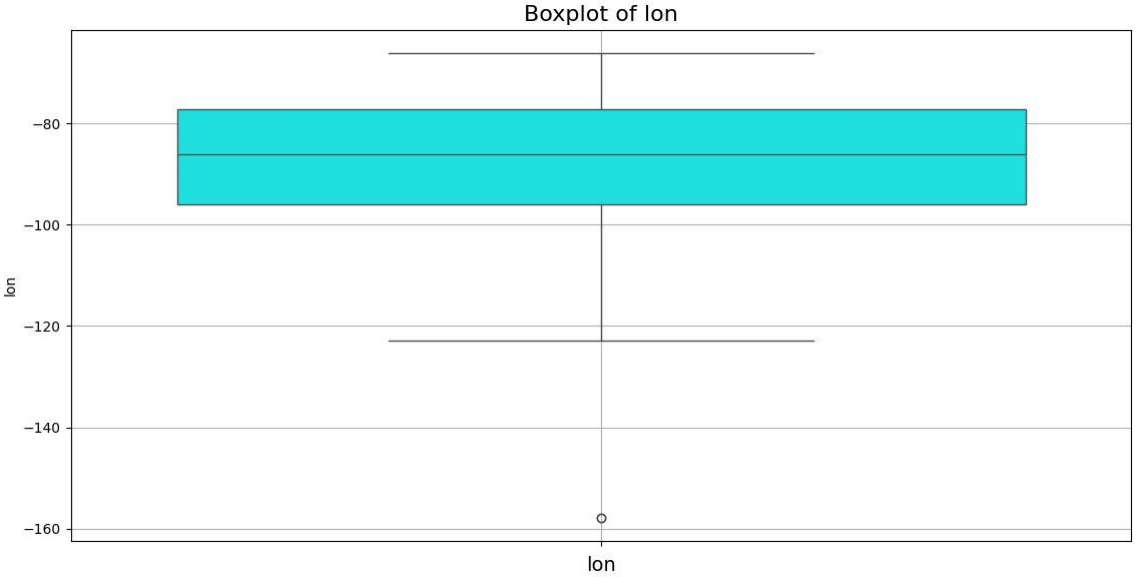
```
import pandas as pd import
matplotlib.pyplot as plt
import seaborn as sns # Load
the dataset
file_name = "D:/Sem 6 Practicals/ABA Practicals/fortune500.csv"  # Replace with your file
path df = pd.read_csv(file_name) # Specify the variable to analyze variable = 'lon'  #
Replace 'lon' with your desired column name
# Check if the variable exists in the dataset if
variable in df.columns:
   # Drop missing values for the selected variable
data = df[variable].dropna()
   # Histogram     plt.figure(figsize=(10, 6))     sns.histplot(data, bins=20, kde=True,
color='blue', label='Histogram with KDE')     plt.axvline(data.mean(), color='red', linestyle='--
', label=f'Mean: {data.mean():.2f}')     plt.axvline(data.median(), color='green', linestyle='--',
label=f'Median: {data.median():.2f}')     plt.title(f"Histogram of {variable}", fontsize=16)
plt.xlabel(variable, fontsize=14)     plt.ylabel("Frequency", fontsize=14)     plt.legend()
plt.grid(True)     plt.show()     # Boxplot

   plt.figure(figsize=(8, 5))     sns.boxplot(data,
color='cyan')     plt.title(f"Boxplot of
{variable}", fontsize=16)     plt.xlabel(variable,
fontsize=14)     plt.grid(True)     plt.show()
   # Analyze characteristics     print(f"Analysis
of {variable}:")     print(f"Mean:
{data.mean():.2f}")     print(f"Median:
```

```
{data.median():.2f}")      print(f"Mode:
{data.mode()[0]:.2f}")      print(f"Minimum:
{data.min():.2f}")      print(f"Maximum:
{data.max():.2f}")      print(f"Range: {data.max()
- data.min():.2f}")      print(f"Variance:
{data.var():.2f}")      print(f"Standard Deviation:
{data.std():.2f}")      print(f"Skewness:
{data.skew():.2f}")      print(f"Kurtosis:
{data.kurt():.2f}") else:
    print(f"Error: The variable '{variable}' does not exist in the dataset.")
```

Output:

```
Analysis of lon:
Mean: -89.89
Median: -86.16
Mode: -74.01
Minimum: -157.86
Maximum: -66.06
Range: 91.80
Variance: 237.60
Standard Deviation: 15.41
Skewness: -1.01
Kurtosis: 0.34
```


Histogram of lon
Mean: -89.89
Median: -86.16
Histogram with KDE
Frequency
lon
200
175
150
125
100
75
50
25
0
−160
−140
−120
−100
−80


Boxplot of lon
lon
−80
−100
−120
−140
−160

Applied Business Analytics # Practical No: 3

Name		**Roll No:**	
Class	T.Y.B.Sc. Data Science	**Seat Number:**	
Subject / Topic	Applied Business Analytics		
Topic	Finding Relationships Among Variables:		

Practical No: 3 Finding Relationships Among Variables:

3(A) Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

Code:

```
# Import necessary libraries

import pandas as pd import

seaborn as sns import

matplotlib.pyplot as plt

# Load the dataset

df = pd.read_csv('D:/Sem 6 Practicals/ABA Practicals/P3/WineQT.csv')  # Ensure this path is
correct

# Display the first few rows of the dataset to understand its

structure print("Dataset Preview:") print(df.head())

# Inspect dataset structure and data

types print("\nDataset Info:")

print(df.info())

# Check for any missing values in the dataset print("\nMissing

Values in the Dataset:") print(df.isnull().sum())

# Handle missing values (if any) by filling with the mean of each column

df.fillna(df.mean(), inplace=True)
```

```
# Compute the correlation matrix using Pearson's method correlation_matrix
= df.corr(method='pearson')
# Print the correlation matrix print("\nCorrelation
Matrix:") print(correlation_matrix)
# Visualize the correlation matrix using a heatmap plt.figure(figsize=(12,
8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True,
linewidths=0.5, cbar_kws={"shrink": 0.75}) plt.title('Correlation Matrix of Wine Quality
Dataset') plt.show()
```

Output:

```
Dataset Preview:
 fixed acidity  volatile acidity  citric acid alcohol  quality  Id
0          7.4            0.70        0.00  ...       9.4        5   0
1          7.8            0.88        0.00  ...       9.8        5   1
2          7.8            0.76        0.04  ...       9.8        5   2
3         11.2            0.28        0.56  ...       9.8        6   3
4          7.4            0.70        0.00  ...       9.4        5   4
[5 rows x 13 columns]
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  -------               ----------------  ------
0   fixed acidity         1143 non-null   float64
1   volatile acidity      1143 non-null   float64
2   citric acid           1143 non-null   float64
```

3 residual sugar 1143 non-null float64

4 chlorides 1143 non-null float64

5 free sulfur dioxide 1143 non-null float64 6 total sulfur dioxide 1143 non-null float64

7 density 1143 non-null float64

8 pH 1143 non-null float64

9 sulphates 1143 non-null float64

10 alcohol 1143 non-null float64

11 quality 1143 non-null int64

12 Id 1143 non-null int64

dtypes: float64(11), int64(2)

memory usage: 116.2 KB

None

Missing Values in the Dataset:

fixed acidity 0

volatile acidity 0

citric acid 0

residual sugar 0 chlorides

0

free sulfur dioxide 0 total

sulfur dioxide 0

density 0 pH

0

sulphates 0

alcohol 0

quality 0 Id

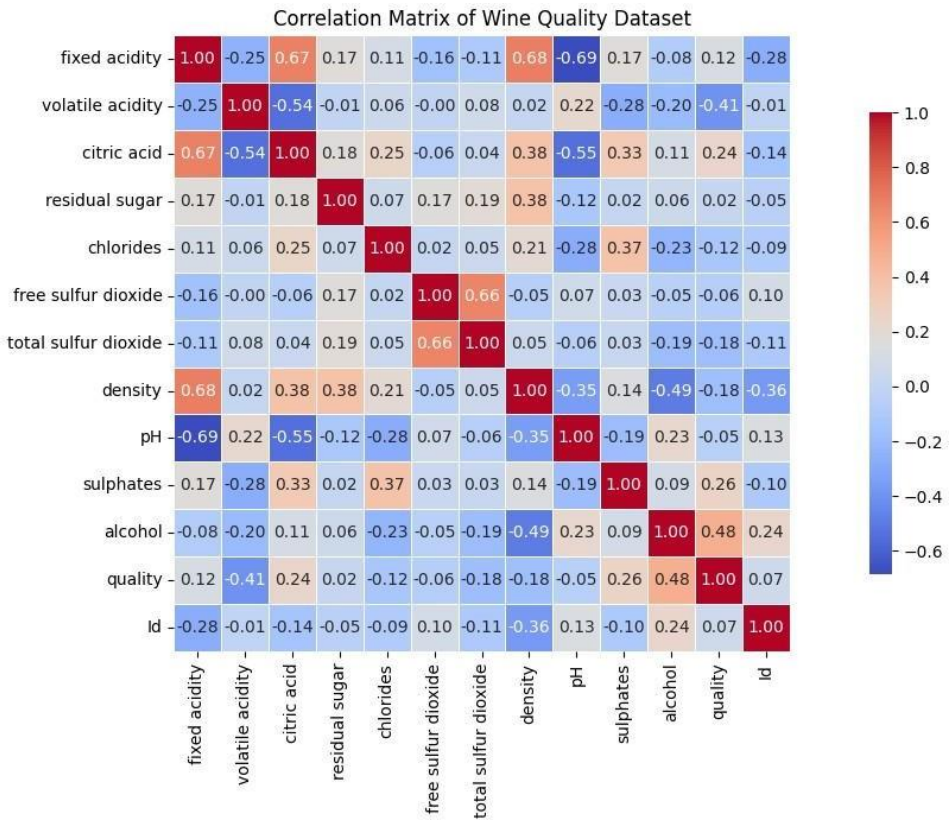
0

dtype: int64

Correlation Matrix:

	fixed acidity	volatile acidity	...	quality	Id
fixed acidity	1.000000	-0.250728	...	0.121970	-0.275826
volatile acidity	-0.250728	1.000000	...	-0.407394	-0.007892
citric acid	0.673157	-0.544187	...	0.240821	-0.139011
residual sugar	0.171831	-0.005751	...	0.022002	-0.046344
chlorides	0.107889	0.056336	...	-0.124085	-0.088099
free sulfur dioxide	-0.164831	-0.001962	...	-0.063260	0.095268
total sulfur dioxide	-0.110628	0.077748	...	-0.183339	-0.107389
density	0.681501	0.016512	...	-0.175208	-0.363926
pH	-0.685163	0.221492	...	-0.052453	0.132904
sulphates	0.174592	-0.276079	...	0.257710	-0.103954
alcohol	-0.075055	-0.203909	...	0.484866	0.238087
quality	0.121970	-0.407394	...	1.000000	0.069708
Id	-0.275826	-0.007892	...	0.069708	1.000000

[13 rows x 13 columns]



3(B) Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

Code:

```
# Import necessary libraries import pandas as pd import
matplotlib.pyplot as plt from sklearn.model_selection
import train_test_split from sklearn.linear_model import
LinearRegression from sklearn.metrics import
mean_squared_error, r2_score
# Load the dataset file_path = 'D:/Sem 6 Practicals/ABA Practicals/P3/Advertising
Budget and Sales.csv' df = pd.read_csv(file_path)
# Independent variable (e.g., TV Ad Budget)
```

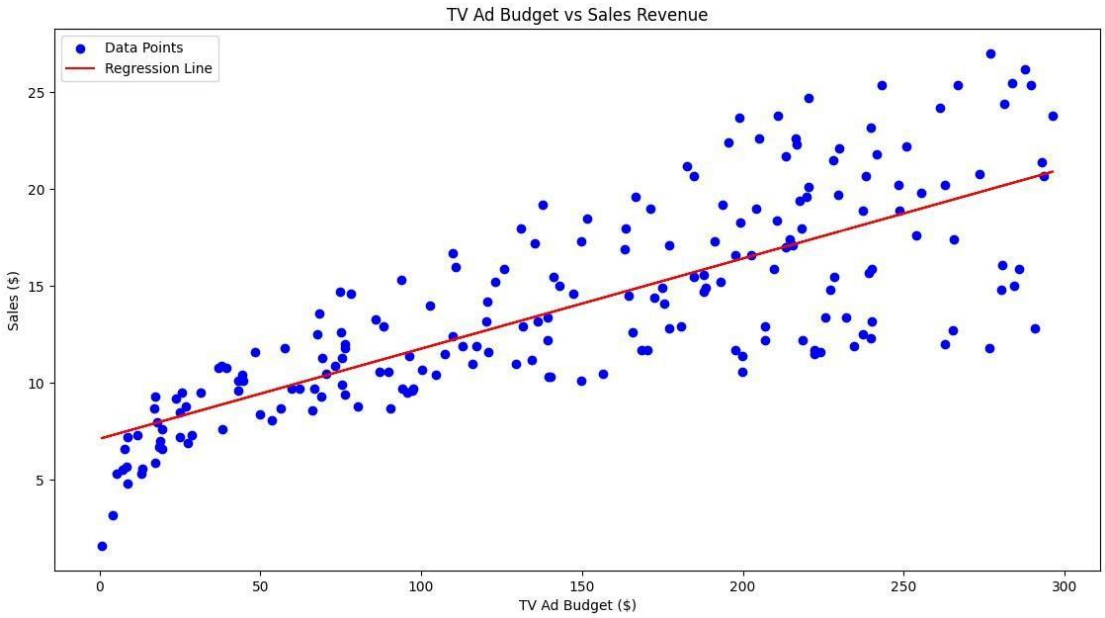
```
X = df[['TV Ad Budget ($)']]
# Dependent variable (Sales Revenue)
y = df['Sales ($)']
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create and fit the Linear Regression model
model = LinearRegression() model.fit(X_train,
y_train) # Predict on the test set y_pred =
model.predict(X_test) # Print model coefficients
print("\nModel Coefficients:") print(f"Intercept:
{model.intercept_}") print(f"Slope (TV Ad
Budget): {model.coef_[0]}") # Evaluate the
model mse = mean_squared_error(y_test,
y_pred) r2 = r2_score(y_test, y_pred)
print("\nModel Evaluation:") print(f"Mean
Squared Error: {mse}") print(f"R^2 Score: {r2}")
# Visualize the regression line plt.scatter(X, y,
color='blue', label='Data Points') plt.plot(X,
model.predict(X), color='red', label='Regression
Line') plt.xlabel("TV Ad Budget ($)")
plt.ylabel("Sales ($)") plt.title("TV Ad Budget vs
Sales Revenue") plt.legend() plt.show()
```

Output:

```
Model Coefficients:
Intercept: 7.119638430592953
Slope (TV Ad Budget): 0.046529733705443346 Model
Evaluation:
```

Mean Squared Error: 10.204654118800956

R^2 Score: 0.6766954295627076



Applied Business Analytics # Practical No: 4

Name		Roll No:	46
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Business Intelligence Tools for Data Analysis:		

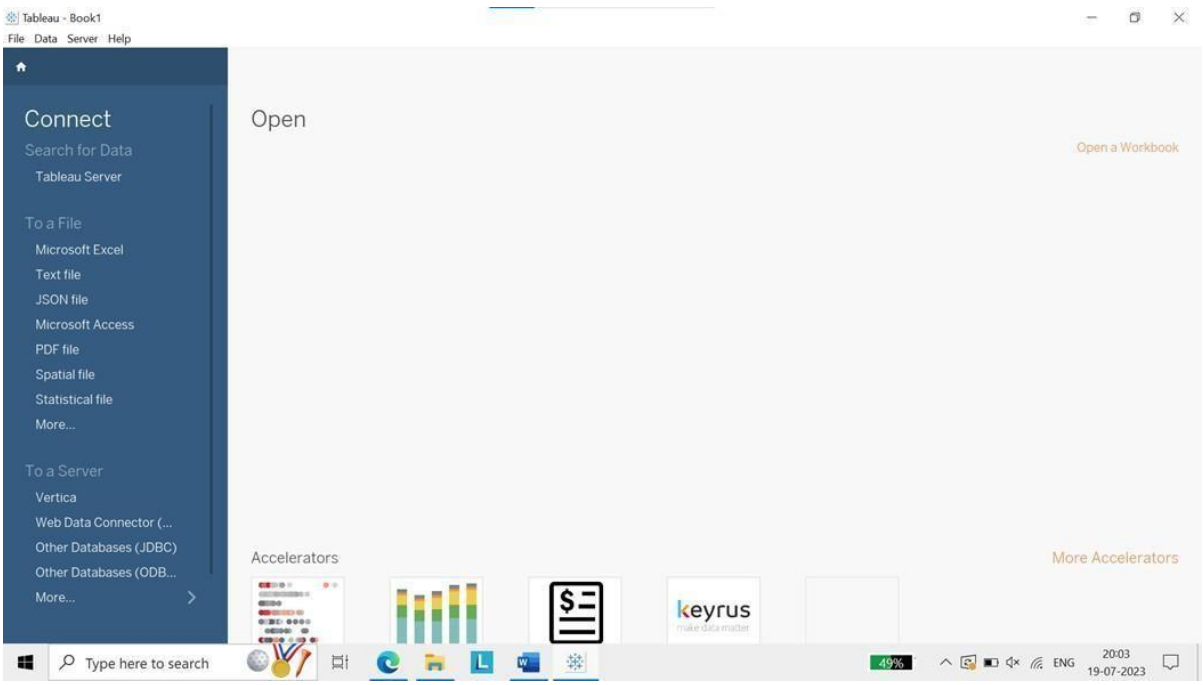
Practical No: 4 Business Intelligence Tools for Data Analysis:

4(A) Utilize a business intelligence tool (e.g., Tableau, Power BI) to extract insights from a dataset and create interactive visualizations for effective data analysis.

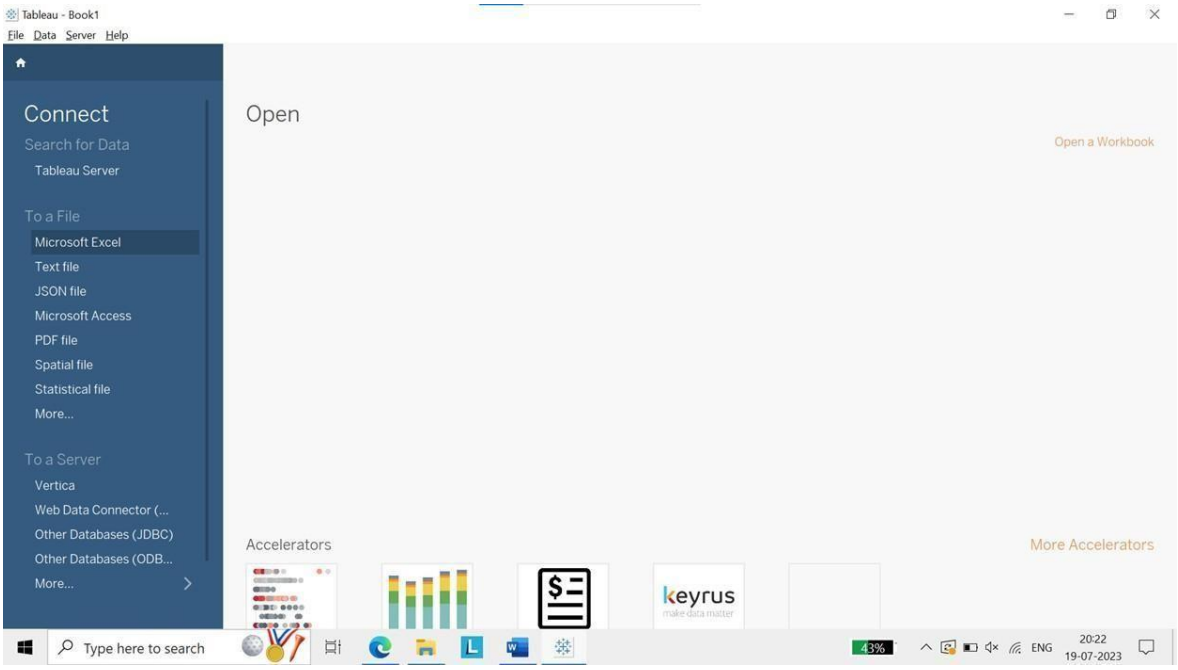
Steps to Perform:

Prepare data

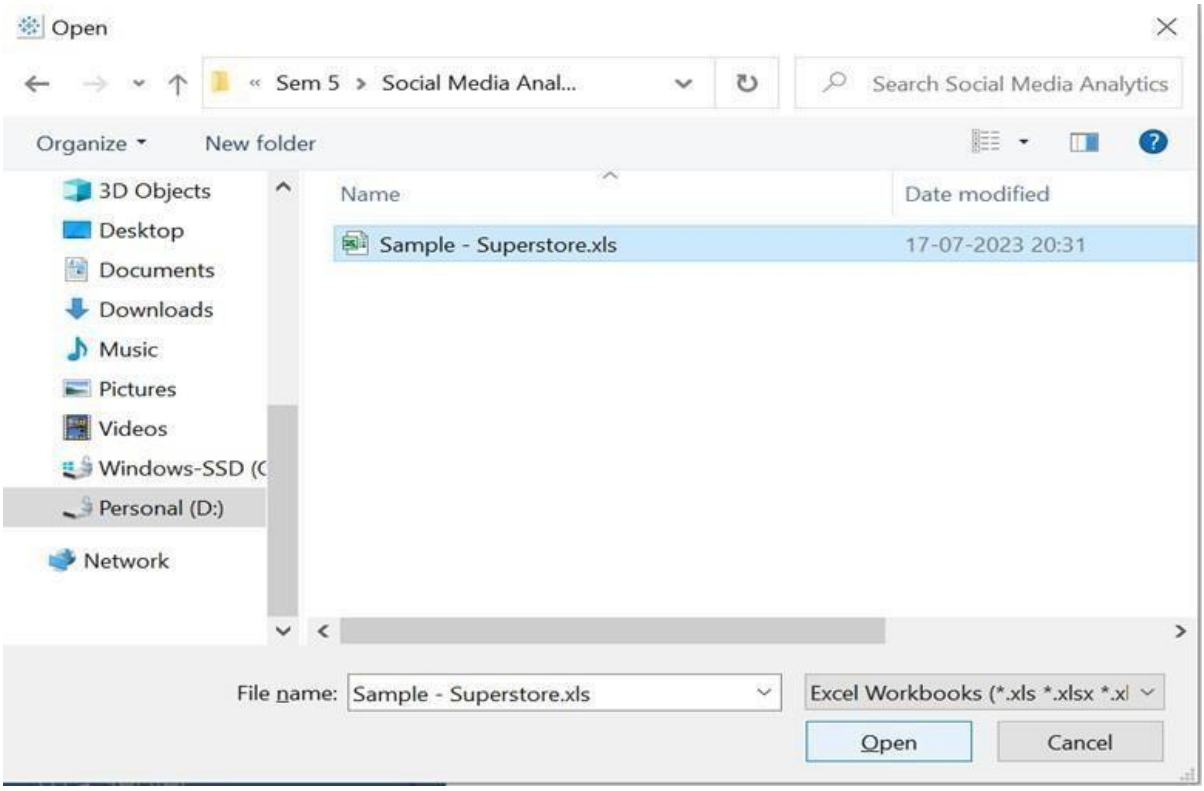
Step 1: Open the Tableau application window.



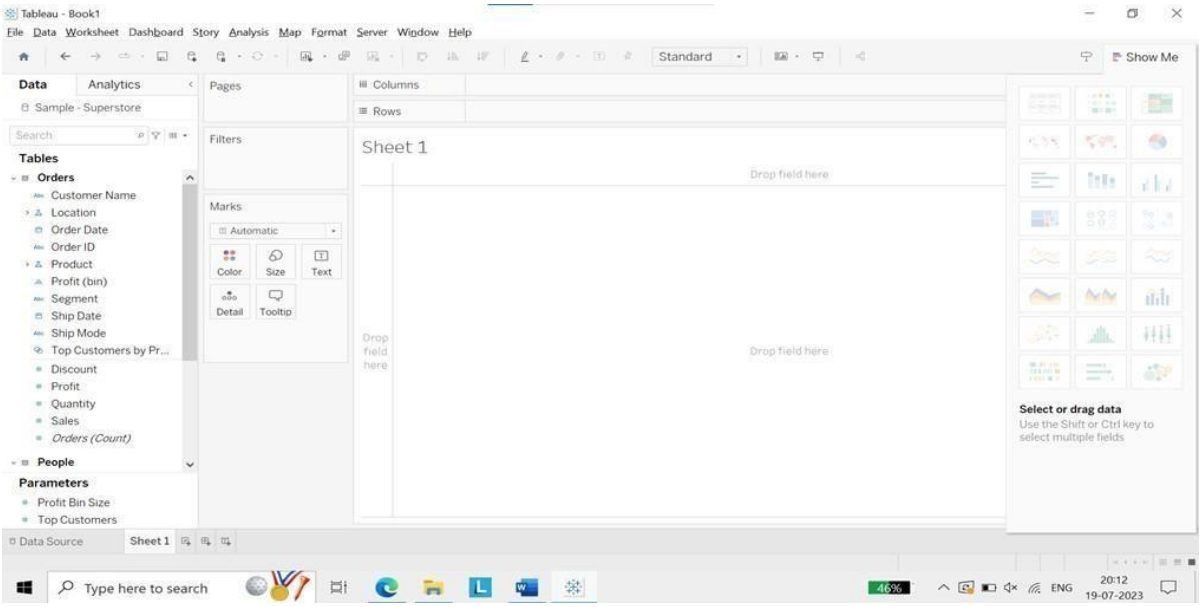
Step 2: In the Connect pane, under the 'To A File' section click on the file type of your desired data file. In this case, we are using an excel file.



Step 3: The 'Open' dialog box appears. Select the location of your desired file and click on 'Open'.

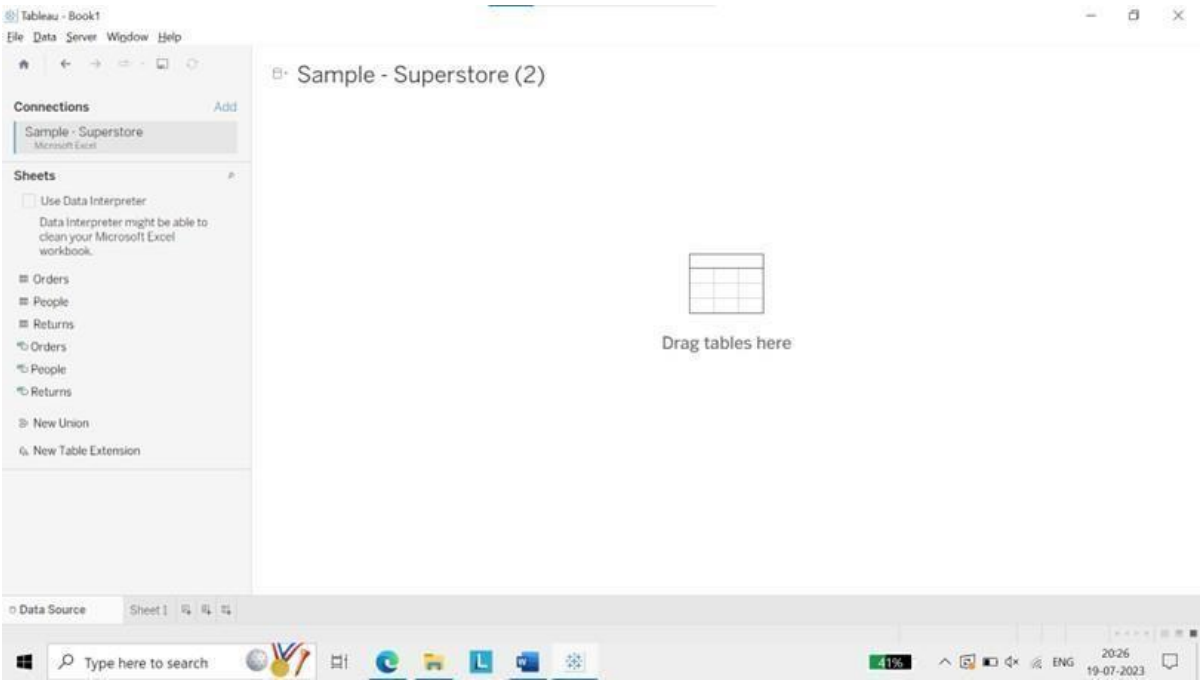


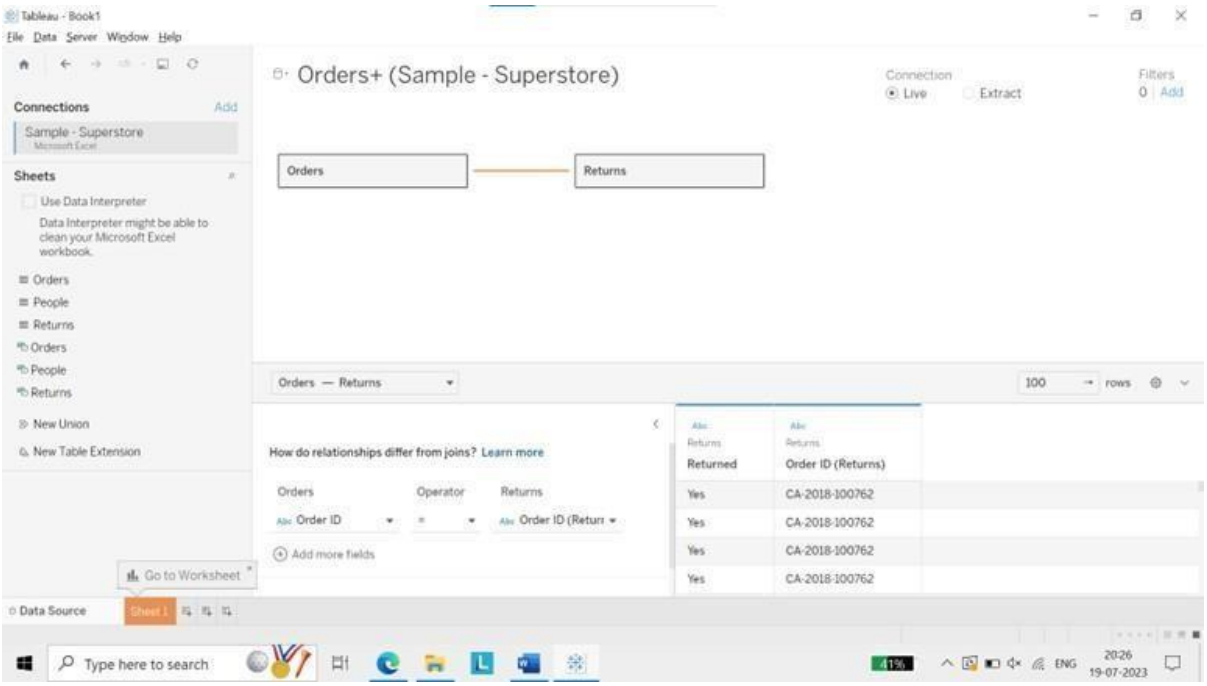
Step 4: The workspace window opens.



Navigate workspace

Step 1: Drag the 'Orders' and 'Returns' columns to the 'Drag Tables Here' section.

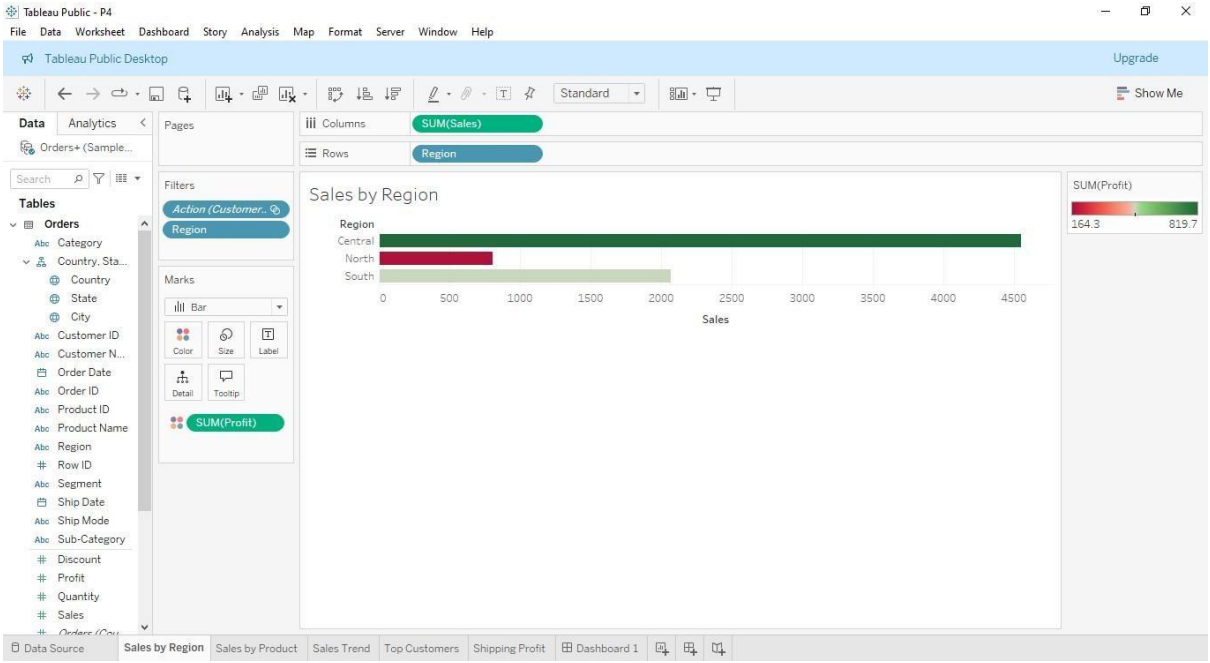




Step 2: Create Visualizations

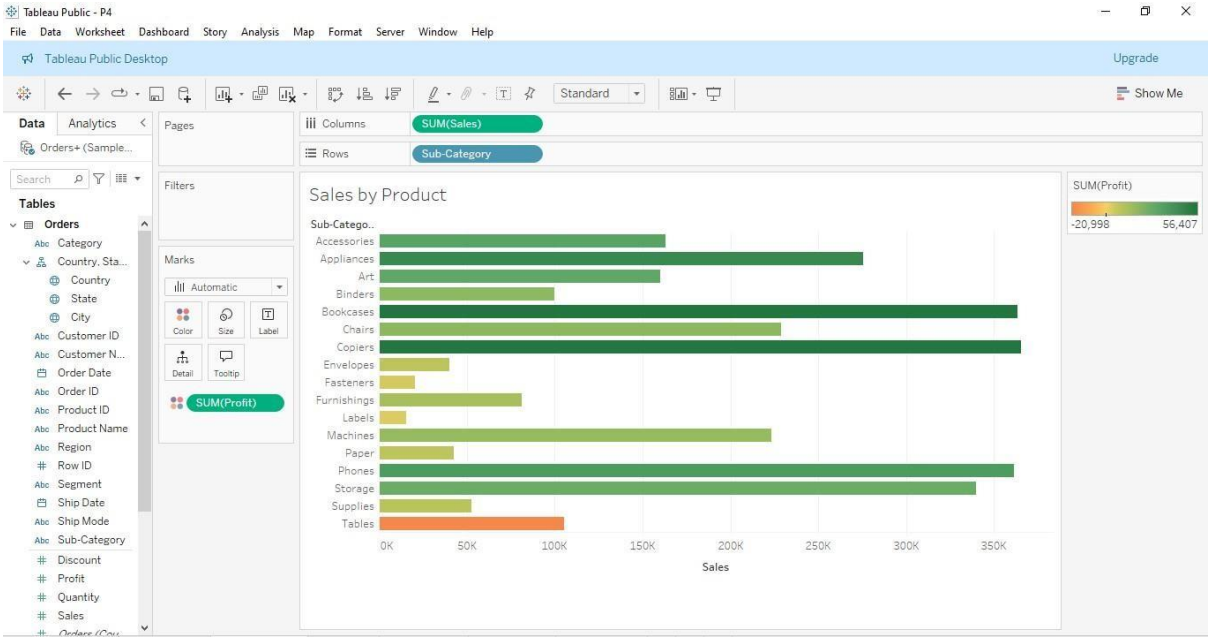
1. Sales and Profit by Region

- **Objective: See which region is most profitable.**
- **Steps:**
  1. **Drag Region to Rows.**
  2. **Drag Sales to Columns.**
  3. **Drag Profit to Color on the Marks card.**
  4. **Change Color to a diverging scale (e.g., green for positive profit, red for negative).**
  5. **Rename the sheet to "Sales by Region" (Right-click on the tab name → Rename).**



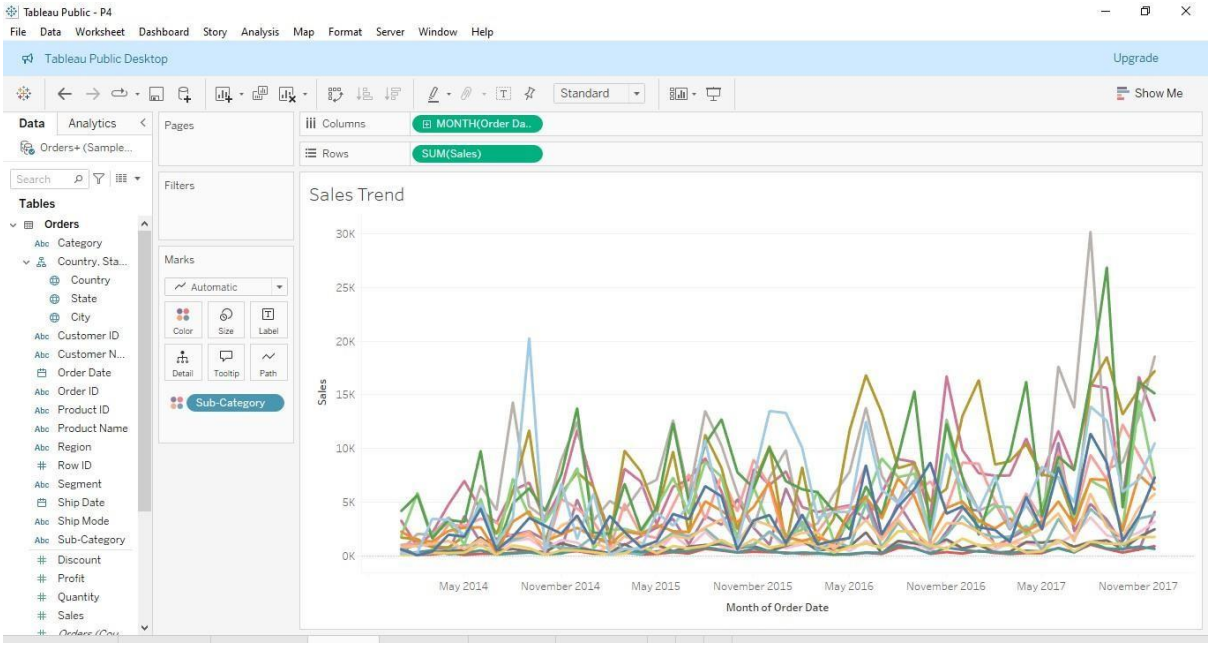
2. Sales by Product Sub-Category

- **Objective: Understand which products are driving sales.**
- **Steps:**
  1. **Drag Sub-Category to Rows.**
  2. **Drag Sales to Columns.**
  3. **Sort the chart by descending sales (click the sort icon above the chart).**
  4. **Drag Profit to Color to see which products are profitable.**
  5. **Rename the sheet to "Sales by Product".**



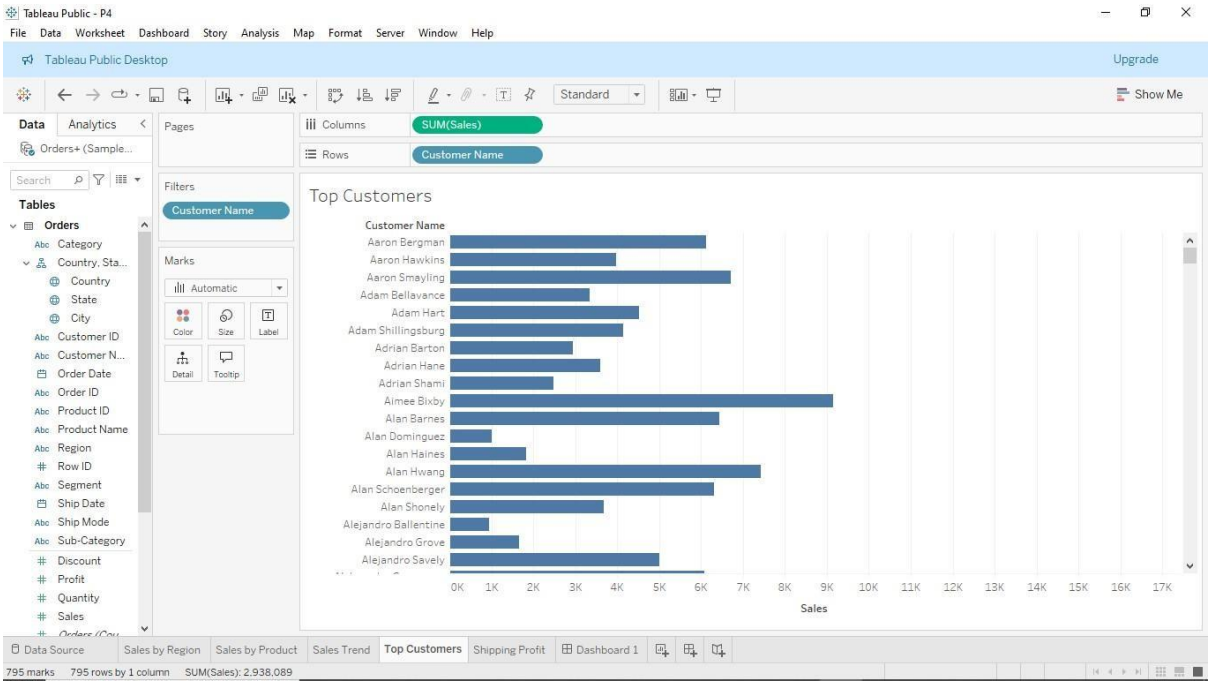
3. Monthly Sales Trend

- **Objective: Analyze how sales change over time.**
- **Steps:**
    1. **Drag Order Date to Columns and change it to Month/Year.**
    2. **Drag Sales to Rows.**
    3. **Change the Marks type to Line for a trend line.**
    4. **Rename the sheet to "Sales Trend".**



4. Top 10 Customers by Sales

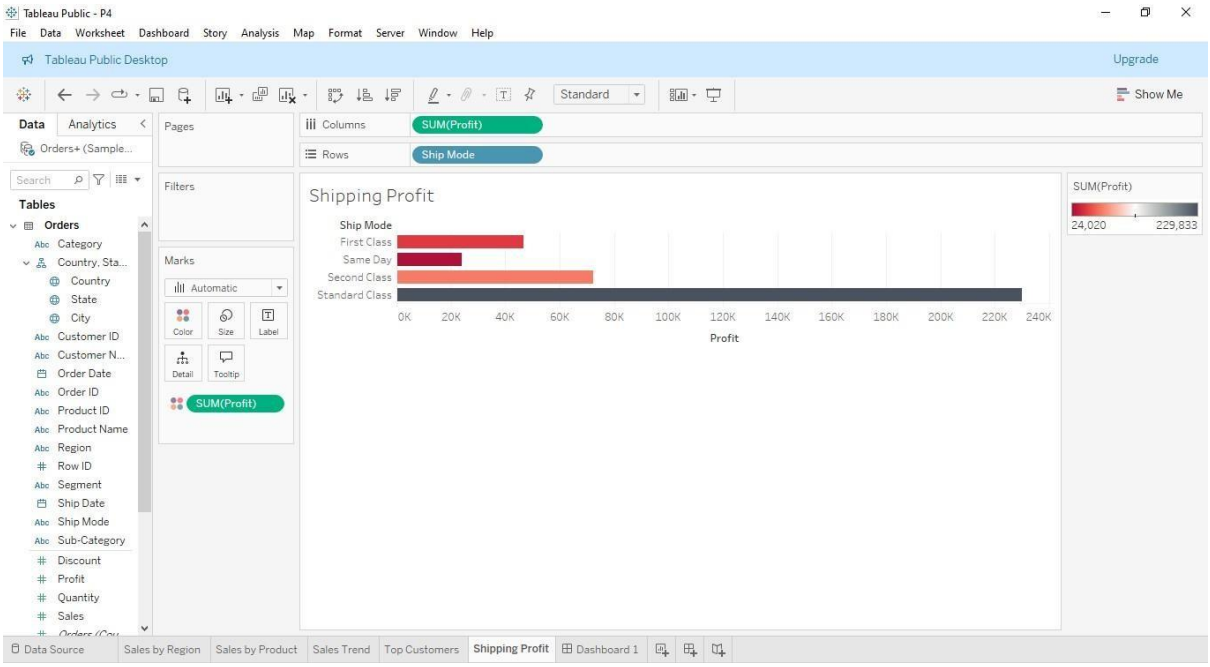
- **Objective: Identify the top 10 customers by sales.**
- **Steps:**
  1. **Drag Customer Name to Rows.**
  2. **Drag Sales to Columns.**
  3. **Sort by descending sales (click the sort icon).**
  4. **Drag Customer Name to Filters, go to the Top tab, and filter for Top 10 by Sales.**
  5. **Rename the sheet to "Top Customers".**



5. Profit by Shipping Mode

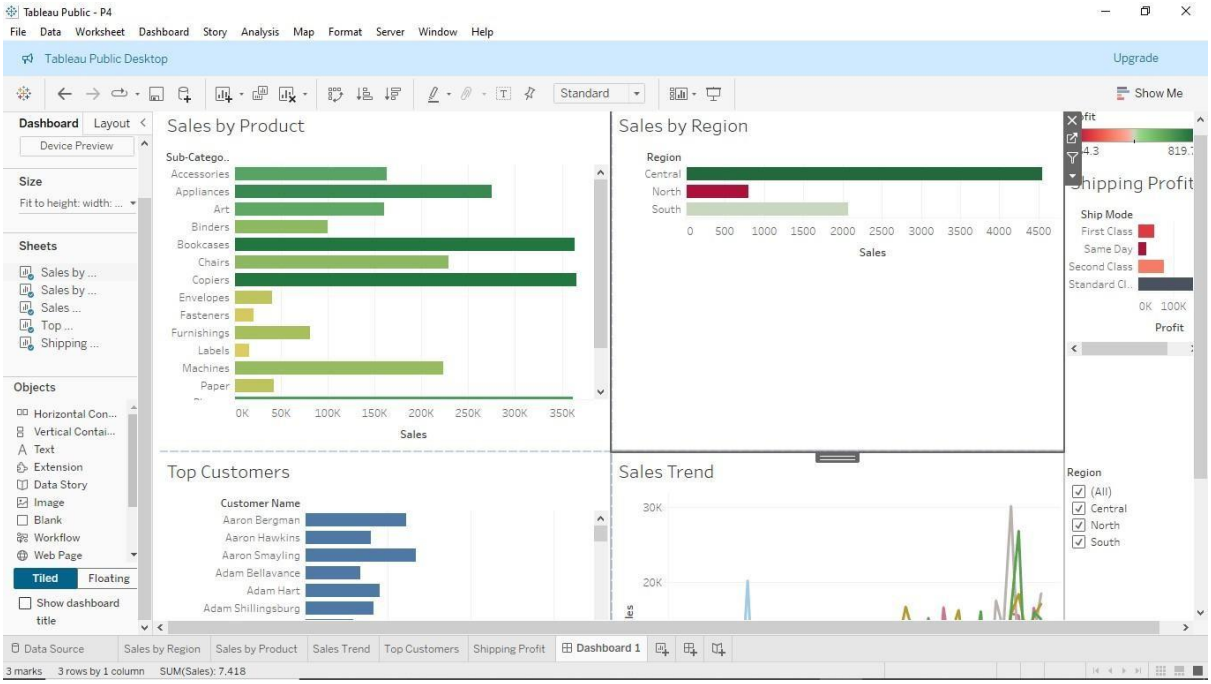
- **Objective: Check how profitable each shipping mode is.**
- **Steps:**

    1. **Drag Ship Mode to Columns.**
    2. **Drag Profit to Rows.**
    3. **Drag Profit to Color on the Marks card.**
    4. **Rename the sheet to "Shipping Profit".**



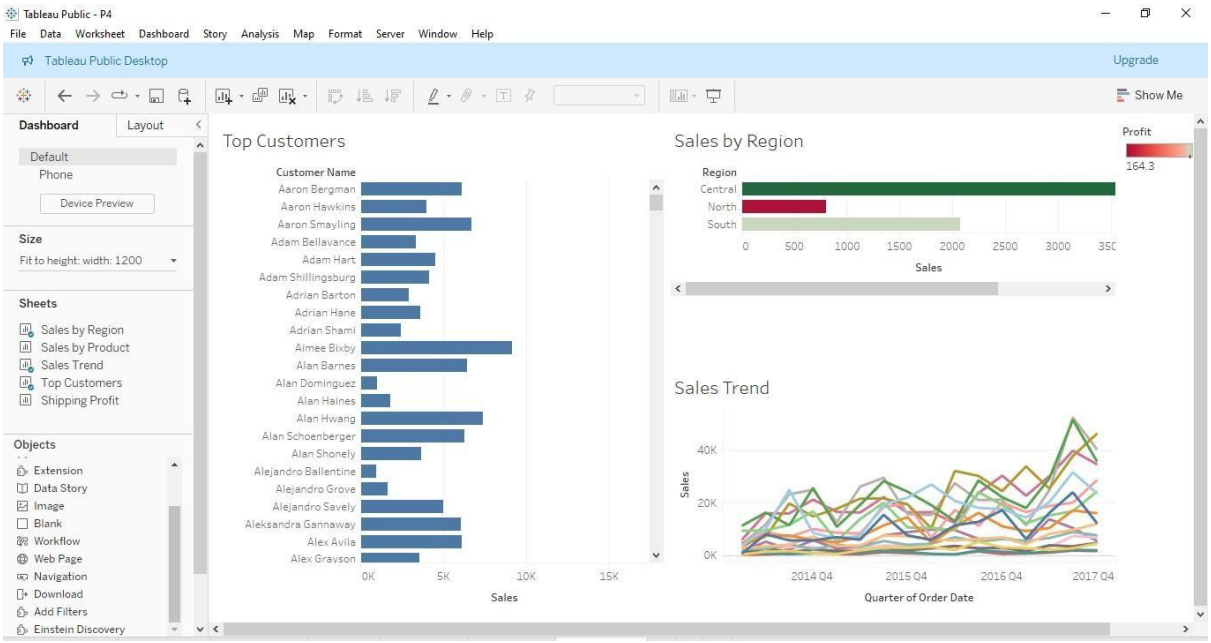
Step 3: Combine Visualizations into a Dashboard

1. **Click the New Dashboard tab at the bottom.**
2. **Drag each sheet (e.g., "Sales by Region", "Top Customers") into the dashboard canvas.**
3. **Add filters for interactivity:**
   - **Drag Category, Region, or Order Date filters from any sheet onto the dashboard. ○ Select Apply to All Worksheets in the filter options for global filters.**
4. **Adjust the layout:**
   - **Resize each chart to fit the space. ○ Use a Text Box (from the objects menu) to add a title or explanation.**



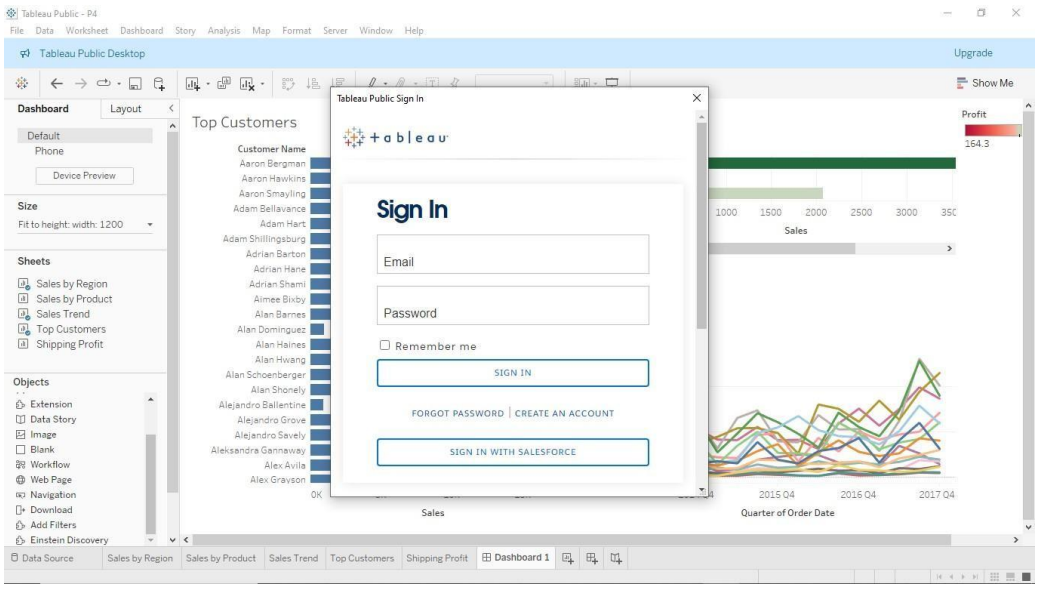
Step 4: Add Interactivity

1. **Add Filter Actions:**
   - **Go to Dashboard → Actions → Add Action → Filter.** ○ **Example: If someone clicks on a region in "Sales by Region," it will highlight relevant data in other charts.**
2. **Enable Tooltips:**
   - **Customize the tooltip in each sheet by clicking on Tooltip in the Marks card.**



Step 5: Save and Share

1. **Save your dashboard:**
   - **Click File → Save As to save it locally.**
2. **Share it:**
   - **Publish to Tableau Public or Tableau Server by clicking File → Publish.**



Applied Business Analytics # Practical No: 5

Name		Roll No:	46
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Probability and Probability Distributions:		

Practical No: 5 Probability and Probability Distributions:

5(A) Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

Code:

```
import random

import matplotlib.pyplot as plt

# Function to simulate the rolling of a six-sided die def

roll_die(num_rolls):

    outcomes = [1, 2, 3, 4, 5, 6]  # Possible outcomes for a fair six-sided die     rolls =

[random.choice(outcomes) for _ in range(num_rolls)]  # Simulate rolling the die

    return rolls

# Simulate rolling the die 10,000 times num_rolls

= 10000

rolls = roll_die(num_rolls)

# Calculate the counts of each outcome outcome_counts = {i: rolls.count(i) for i in range(1,

7)}  # Count occurrences of each outcome

# Calculate the probabilities of each outcome

probabilities = {i: count / num_rolls for i, count in outcome_counts.items()}  # Calculate
probabilities

# Print the probabilities of each outcome print("Probabilities

of each outcome after", num_rolls, "rolls:") for outcome, prob
```

```
in probabilities.items():     print(f"Outcome {outcome}:
{prob:.4f}")

# Visualize the probabilities using a bar plot
plt.bar(probabilities.keys(), probabilities.values(), color='blue')
plt.xlabel('Outcome') plt.ylabel('Probability')
plt.title('Probabilities of Outcomes for Rolling a Six-Sided Die')
plt.xticks(range(1, 7)) plt.show()
```

Output:

Probabilities of each outcome after 10000 rolls:

Outcome 1: 0.1657

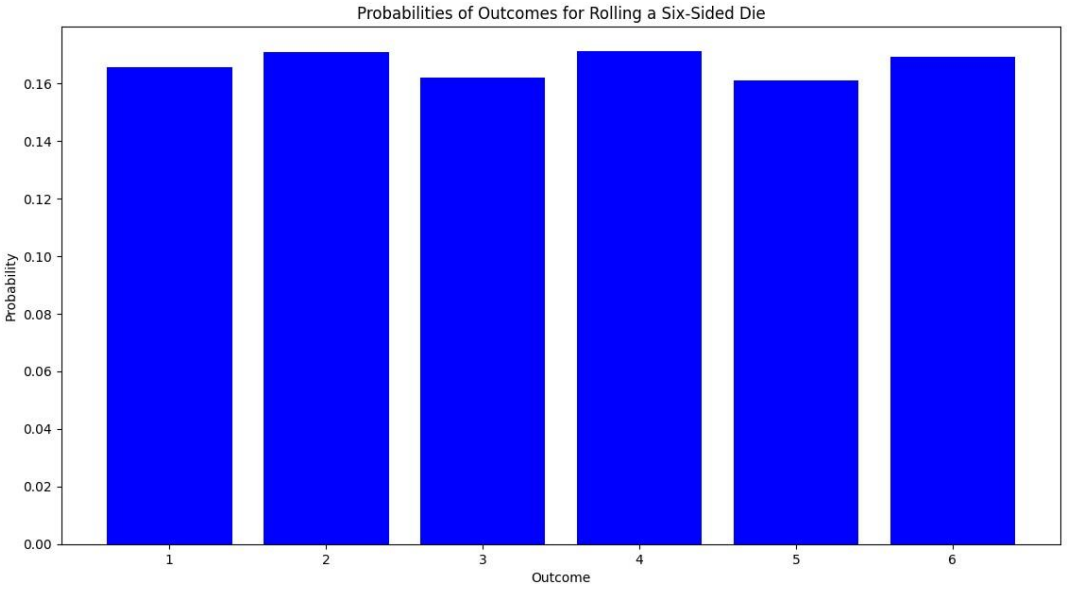
Outcome 2: 0.1709

Outcome 3: 0.1619

Outcome 4: 0.1711

Outcome 5: 0.1612 Outcome

6: 0.1692



5(B) Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

Code:

```
import numpy as np import

matplotlib.pyplot as plt import

seaborn as sns

# Number of random numbers to generate num_samples

= 10000

# Generate random numbers from different distributions

normal_data = np.random.normal(loc=0, scale=1, size=num_samples)   # Normal distribution
(mean=0, std dev=1)

uniform_data = np.random.uniform(low=0, high=10, size=num_samples)  # Uniform
distribution (range [0, 10])

exponential_data = np.random.exponential(scale=1, size=num_samples)  # Exponential
distribution (lambda=1)

# Analyze properties: Mean, Standard Deviation, Skewness, Kurtosis

print("Statistical Properties:")

print(f"Normal Distribution: Mean={np.mean(normal_data):.2f}, Std
Dev={np.std(normal_data):.2f}")

print(f"Uniform Distribution: Mean={np.mean(uniform_data):.2f}, Std
Dev={np.std(uniform_data):.2f}")

print(f"Exponential Distribution: Mean={np.mean(exponential_data):.2f}, Std
Dev={np.std(exponential_data):.2f}")

# Plot the distributions plt.figure(figsize=(15, 10)) # Normal

Distribution Plot plt.subplot(3, 1, 1) sns.histplot(normal_data,

kde=True, bins=50, color='blue', alpha=0.6) plt.title("Normal

Distribution (loc=0, scale=1)")

plt.xlabel("Value") plt.ylabel("Frequency") # Uniform Distribution Plot

plt.subplot(3, 1, 2) sns.histplot(uniform_data, kde=True, bins=50,
```

```
color='green', alpha=0.6) plt.title("Uniform Distribution (low=0, high=10)")
plt.xlabel("Value") plt.ylabel("Frequency") # Exponential Distribution Plot
plt.subplot(3, 1, 3) sns.histplot(exponential_data, kde=True, bins=50,
color='orange', alpha=0.6) plt.title("Exponential Distribution (scale=1)")
plt.xlabel("Value")  plt.ylabel("Frequency")
# Adjust layout to make space for the plots
plt.tight_layout() # Show plots plt.show()
```

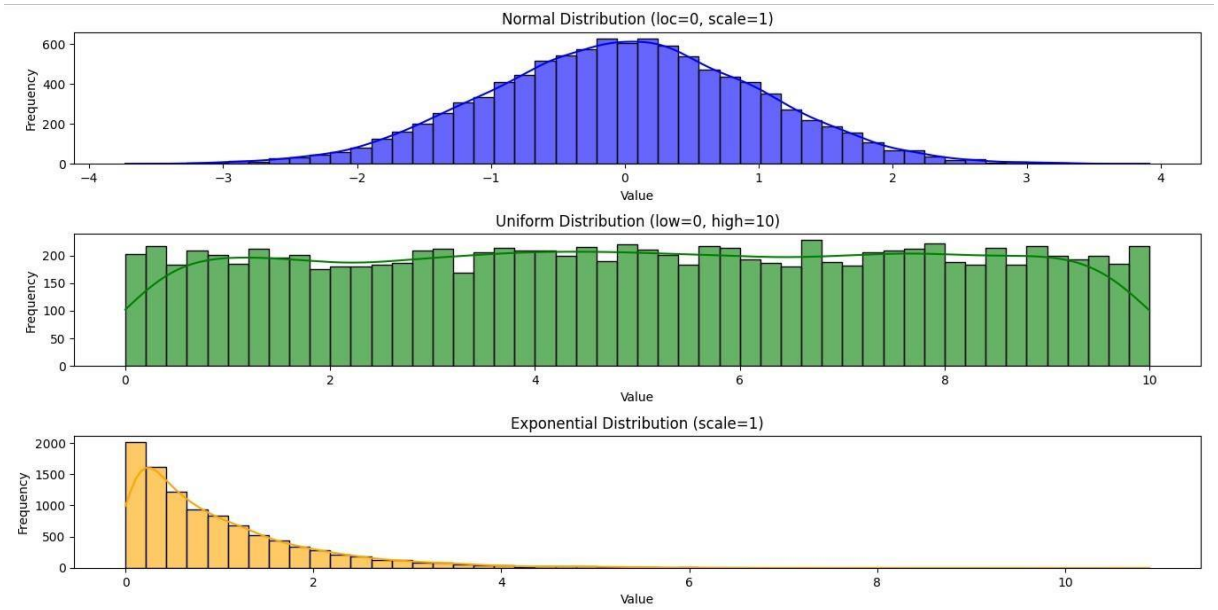
Output:

Statistical Properties:

Normal Distribution: Mean=-0.01, Std Dev=1.00

Uniform Distribution: Mean=5.02, Std Dev=2.88

Exponential Distribution: Mean=1.00, Std Dev=1.01



Applied Business Analytics # Practical No: 6

Name		Roll No:	46
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Decision Making under Uncertainty:		

Practical No: 6 Decision Making under Uncertainty:

6(A) Develop a decision tree model to make business decisions considering uncertainties and associated probabilities at each decision point.

Code:

```
# Import necessary libraries import pandas as pd from sklearn.model_selection

import train_test_split from sklearn.tree import DecisionTreeClassifier, export_text,

plot_tree from sklearn.metrics import classification_report, confusion_matrix,

accuracy_score import matplotlib.pyplot as plt # Define main function def main():

    # File path for the dataset

    file_path = r"D:/Sem 6 Practicals/ABA Practicals/P6/adult.csv"  # Replace with your CSV file path

    # Define column names     column_names = ['age', 'workclass', 'fnlwgt',

'education', 'education-num',

                'marital-status', 'occupation', 'relationship', 'race',

                'sex', 'capital-gain', 'capital-loss', 'hours-per-week',

                'native-country', 'income']

    # Load the dataset

    data = pd.read_csv(file_path, header=None, names=column_names, na_values=' ?', skipinitialspace=True)

    # Preprocess the data: drop rows with missing values

data.dropna(inplace=True)
```

```
# Convert categorical variables to numerical using one-hot encoding
data = pd.get_dummies(data, drop_first=True)  # Define features (X)
and target variable (y)  X = data.drop('income_>50K', axis=1)  #
Features  y = data['income_>50K']  # Target variable (1 if income
>50K, else 0)
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train a Decision Tree model  model = DecisionTreeClassifier(criterion='gini',
max_depth=5, random_state=42)  model.fit(X_train, y_train)  # Evaluate the
model  y_pred = model.predict(X_test)  accuracy = accuracy_score(y_test,
y_pred)  print("Model Accuracy:", accuracy)  print("\nClassification Report:\n",
classification_report(y_test, y_pred))  print("\nConfusion Matrix:\n",
confusion_matrix(y_test, y_pred))
# Visualize the decision tree
plt.figure(figsize=(20, 10))
plot_tree(model, feature_names=X.columns, class_names=["<=50K", ">50K"], filled=True,
rounded=True)
plt.title("Decision Tree for Adult Income Dataset")
plt.show()
# Display decision rules  rules = export_text(model,
feature_names=list(X.columns))  print("\nDecision Tree
Rules:\n", rules)  # Use probabilities for decision-making

probabilities = model.predict_proba(X_test)
X_test_sample = X_test.head(5)  # Analyze probabilities for 5 test samples
sample_probabilities = probabilities[:5]
# Display probabilities
decision_samples = pd.DataFrame({
    "Sample": range(1, 6),
    "Probability <=50K": [p[0] for p in sample_probabilities],
    "Probability >50K": [p[1] for p in sample_probabilities]
```

```
    })
    print("\nSample Decision Probabilities:")
print(decision_samples) # Run the main
function if __name___== "__main__":
    main()
```

Output:

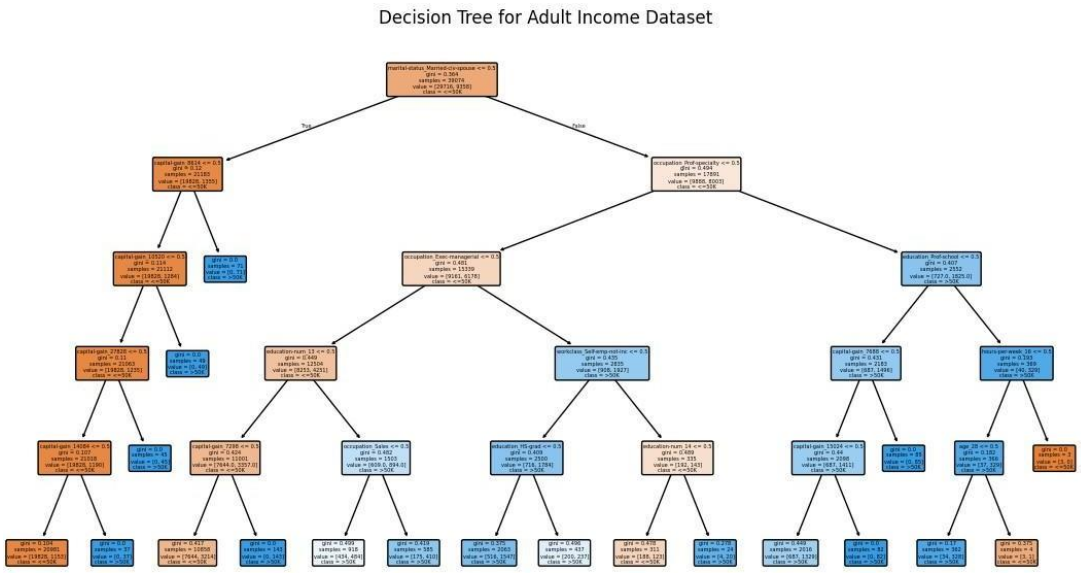
```
Model Accuracy: 0.8319172893847886
Classification Report:
              precision    recall  f1-score   support

       False       0.86      0.93      0.89      7440
        True       0.70      0.52      0.60      2329

    accuracy                           0.83      9769
macro avg       0.78      0.73      0.75      9769
weighted avg       0.82      0.83      0.82      9769
Confusion Matrix:
 [[6910  530]
[1112 1217]]
Decision Tree Rules:
 |--- marital-status_Married-civ-spouse <= 0.50
|   |--- capital-gain_8614 <= 0.50
|   |   |--- capital-gain_10520 <= 0.50
|   |   |   |--- capital-gain_27828 <= 0.50
|   |   |   |   |--- capital-gain_14084 <= 0.50
|   |   |   |   |   |--- class: False
|   |   |   |   |--- capital-gain_14084 >  0.50
```

```
|   |   |   |   |   |--- class: True
|   |   |   |--- capital-gain_27828 >  0.50
|   |   |   |   |--- class: True
|   |   |--- capital-gain_10520 >  0.50
|   |   |   |--- class: True
|   |--- capital-gain_8614 >  0.50
|   |   |--- class: True
|--- marital-status_Married-civ-spouse >  0.50
|   |--- occupation_Prof-specialty <= 0.50
|   |   |--- occupation_Exec-managerial <= 0.50
|   |   |   |--- education-num_13 <= 0.50
|   |   |   |   |--- capital-gain_7298 <= 0.50
|   |   |   |   |   |--- class: False
|   |   |   |   |--- capital-gain_7298 >  0.50
|   |   |   |   |   |--- class: True
|   |   |   |--- education-num_13 >  0.50
|   |   |   |   |--- occupation_Sales <= 0.50
|   |   |   |   |   |--- class: True
|   |   |   |   |--- occupation_Sales >  0.50
|   |   |   |   |   |--- class: True
|   |   |--- occupation_Exec-managerial >  0.50
|   |   |   |--- workclass_Self-emp-not-inc <= 0.50
|   |   |   |   |--- education_HS-grad <= 0.50
|   |   |   |   |   |--- class: True
|   |   |   |   |--- education_HS-grad >  0.50
|   |   |   |   |   |--- class: True
|   |   |   |--- workclass_Self-emp-not-inc >  0.50
|   |   |   |   |--- education-num_14 <= 0.50
|   |   |   |   |   |--- class: False
|   |   |   |   |--- education-num_14 >  0.50
```

```
|   |   |   |   |   |--- class: True
|   |--- occupation_Prof-specialty >  0.50 |
|   |--- education_Prof-school <= 0.50
|   |   |   |--- capital-gain_7688 <= 0.50
|   |   |   |   |--- capital-gain_15024 <= 0.50
|   |   |   |   |   |--- class: True
|   |   |   |   |--- capital-gain_15024 >  0.50
|   |   |   |   |   |--- class: True
|   |   |   |--- capital-gain_7688 >  0.50
|   |   |   |   |--- class: True
|   |   |--- education_Prof-school >  0.50
|   |   |   |--- hours-per-week_16 <= 0.50
|   |   |   |   |--- age_28 <= 0.50
|   |   |   |   |   |--- class: True
|   |   |   |   |--- age_28 >  0.50
|   |   |   |   |   |--- class: False
|   |   |   |--- hours-per-week_16 >  0.50
|   |   |   |   |--- class: False
```

Sample Decision Probabilities:

Sample	Probability <=50K	Probability >50K
0	1	0.945046	0.054954
1	2	0.250121	0.749879
2	3	0.703997	0.296003
3	4	0.945046	0.054954
4	5	0.945046	0.054954



6(B) Apply the concept of expected value to evaluate different decision alternatives and select the optimal one.

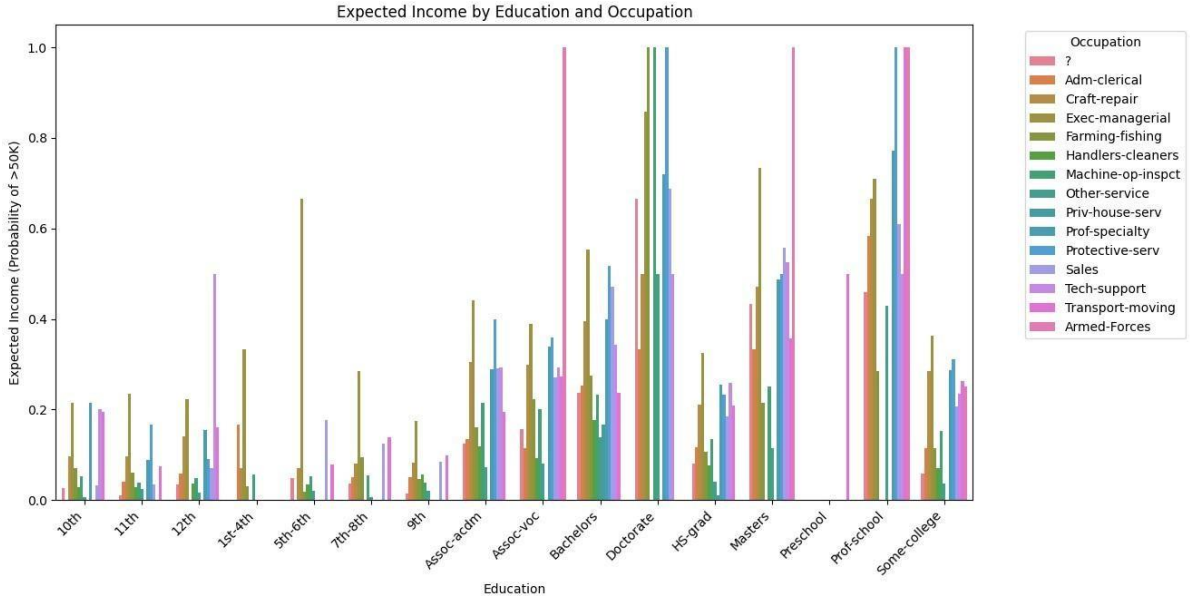
Code:

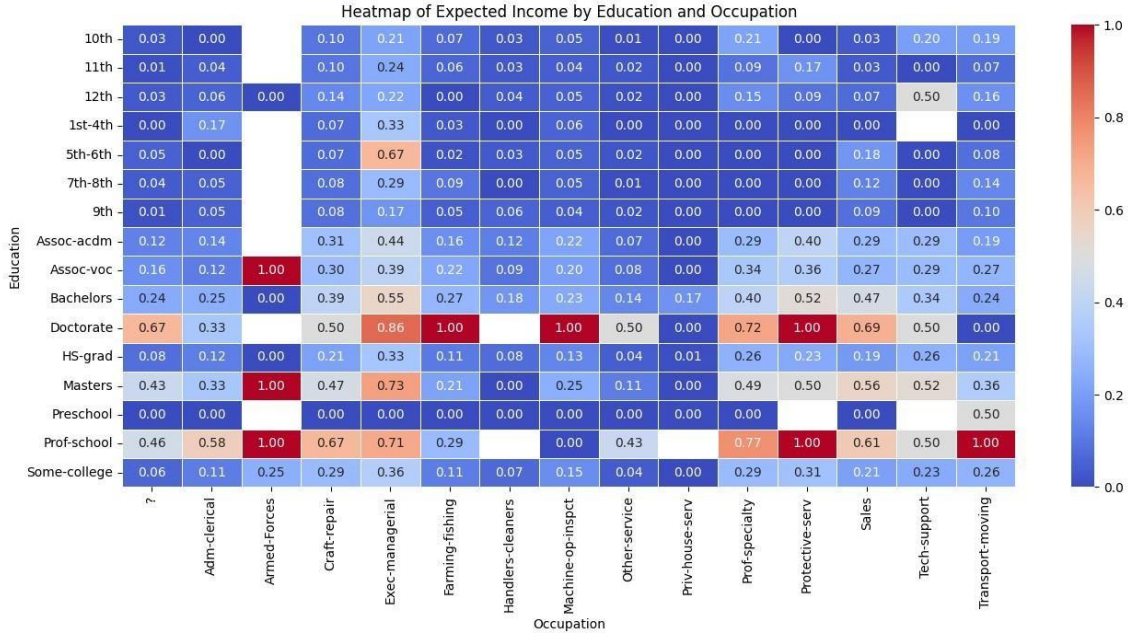
```
import pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns
# Load the dataset
# Make sure to have the 'adult.csv' file in your working directory data
= pd.read_csv('D:/Sem 6 Practicals/ABA Practicals/P6/adult.csv')
# Display the first few rows of the dataset print("First
few rows of the dataset:") print(data.head())
# Convert income to binary values for calculation (0 for <=50K, 1 for >50K) data['income']
= data['income'].map({'<=50K': 0, '>50K': 1})
```

```
# Group by education and occupation, calculating mean income (which represents expected
value) income_summary = data.groupby(['education',
'occupation'])['income'].mean().reset_index()
# Rename columns for clarity income_summary.columns = ['education',
'occupation', 'expected_income']
# Display the expected income summary print("\nExpected
Income Summary:") print(income_summary)
# Find the combination with the highest expected income (i.e., the optimal decision
alternative)
optimal_decision = income_summary.loc[income_summary['expected_income'].idxmax()]
# Display the optimal decision alternative print(f"\nOptimal Decision: Education -
{optimal_decision['education']}, "       f"Occupation -
{optimal_decision['occupation']}, "       f"Expected Income -
{optimal_decision['expected_income'] * 100:.2f}% >50K") # Visualization 1: Bar
Plot of Expected Income by Education and Occupation

plt.figure(figsize=(12, 6))
sns.barplot(x='education', y='expected_income', hue='occupation', data=income_summary)
plt.title('Expected Income by Education and Occupation') plt.xlabel('Education')
plt.ylabel('Expected Income (Probability of >50K)') plt.xticks(rotation=45, ha='right')
plt.legend(title='Occupation', bbox_to_anchor=(1.05, 1), loc='upper left') plt.tight_layout()
plt.show()
# Visualization 2: Heatmap of Expected Income by Education and Occupation
# Pivot the data to create a matrix for the heatmap
pivot_data = income_summary.pivot(index='education', columns='occupation',
values='expected_income')
plt.figure(figsize=(12, 8))
sns.heatmap(pivot_data, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Heatmap of Expected Income by Education and Occupation')
plt.xlabel('Occupation') plt.ylabel('Education') plt.tight_layout() plt.show()
```

Output:

```
First few rows of the dataset:   age  workclass  fnlwgt  ... hours-
per-week  native-country income
0   25    Private  226802  ...          40   United-States  <=50K
1   38    Private   89814  ...          50   United-States  <=50K
2   28  Local-gov  336951  ...          40   United-States   >50K
3   44    Private  160323  ...          40   United-States   >50K
4   18          ?  103497  ...          30   United-States  <=50K

[5 rows x 15 columns]
Expected Income Summary:
       education       occupation  expected_income
0           10th                ?         0.026667
1           10th     Adm-clerical         0.000000
2           10th     Craft-repair         0.096234
3           10th   Exec-managerial         0.214286
4           10th   Farming-fishing         0.070423
..           ...              ...              ...
220  Some-college    Prof-specialty         0.286385
221  Some-college   Protective-serv         0.311688
222  Some-college            Sales         0.206919
223  Some-college      Tech-support         0.234742
224  Some-college  Transport-moving         0.263415
[225 rows x 3 columns]
Optimal Decision: Education - Assoc-voc, Occupation - Armed-Forces, Expected Income -
100.00% >50K
```


Expected Income by Education and Occupation
Expected Income (Probability of >50K)
Education
10th
11th
12th
1st-4th
5th-6th
7th-8th
9th
Assoc-acdm
Assoc-voc
Bachelors
Doctorate
HS-grad
Masters
Preschool
Prof-school
Some-college
Occupation
?
Adm-clerical
Craft-repair
Exec-managerial
Farming-fishing
Handlers-cleaners
Machine-op-inspct
Other-service
Priv-house-serv
Prof-specialty
Protective-serv
Sales
Tech-support
Transport-moving
Armed-Forces


Heatmap of Expected Income by Education and Occupation
Education
Occupation
10th 0.03 0.00 0.10 0.21 0.07 0.03 0.05 0.01 0.00 0.21 0.00 0.03 0.20 0.19
11th 0.01 0.04 0.10 0.24 0.06 0.03 0.04 0.02 0.00 0.09 0.17 0.03 0.00 0.07
12th 0.03 0.06 0.00 0.14 0.22 0.00 0.04 0.05 0.02 0.00 0.15 0.09 0.07 0.50 0.16
1st-4th 0.00 0.17 0.07 0.33 0.03 0.00 0.06 0.00 0.00 0.00 0.00 0.00 0.00
5th-6th 0.05 0.00 0.07 0.67 0.02 0.03 0.05 0.02 0.00 0.00 0.00 0.18 0.00 0.08
7th-8th 0.04 0.05 0.08 0.29 0.09 0.00 0.05 0.01 0.00 0.00 0.00 0.12 0.00 0.14
9th 0.01 0.05 0.08 0.17 0.05 0.06 0.04 0.02 0.00 0.00 0.00 0.09 0.00 0.10
Assoc-acdm 0.12 0.14 0.31 0.44 0.16 0.12 0.22 0.07 0.00 0.29 0.40 0.29 0.29 0.19
Assoc-voc 0.16 0.12 1.00 0.30 0.39 0.22 0.09 0.20 0.08 0.00 0.34 0.36 0.27 0.29 0.27
Bachelors 0.24 0.25 0.00 0.39 0.55 0.27 0.18 0.23 0.14 0.17 0.40 0.52 0.47 0.34 0.24
Doctorate 0.67 0.33 0.50 0.86 1.00 1.00 0.50 0.00 0.72 1.00 0.69 0.50 0.00
HS-grad 0.08 0.12 0.00 0.21 0.33 0.11 0.08 0.13 0.04 0.01 0.26 0.23 0.19 0.26 0.21
Masters 0.43 0.33 1.00 0.47 0.73 0.21 0.00 0.25 0.11 0.00 0.49 0.50 0.56 0.52 0.36
Preschool 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.50
Prof-school 0.46 0.58 1.00 0.67 0.71 0.29 0.00 0.43 0.77 1.00 0.61 0.50 1.00
Some-college 0.06 0.11 0.25 0.29 0.36 0.11 0.07 0.15 0.04 0.00 0.29 0.31 0.21 0.23 0.26
?
Adm-clerical
Armed-Forces
Craft-repair
Exec-managerial
Farming-fishing
Handlers-cleaners
Machine-op-inspct
Other-service
Priv-house-serv
Prof-specialty
Protective-serv
Sales
Tech-support
Transport-moving

Applied Business Analytics # Practical No: 7

Name		Roll No:	46
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Sampling and Sampling Distributions:		

Practical No: 7 Sampling and Sampling Distributions:

7(A) Conduct a survey and collect data from a sample population, ensuring proper sampling techniques are employed.

Code:

```
import pandas as pd import

numpy as np

# Step 1: Load the data from the CSV file data = pd.read_csv('D:/Sem

6 Practicals/ABA Practicals/P7/bank.csv')

# Step 2: Display the first few rows of the dataset

print("First 5 rows of the dataset:") print(data.head())

# Step 3: Define sampling techniques

# Example: Simple random sampling without replacement sample_size

= 100  # Define your sample size

sampled_data = data.sample(n=sample_size, random_state=1)  # Set random_state for
reproducibility

# Step 4: Display the sampled data

print("\nSampled Data:") print(sampled_data)

# Step 5: Perform basic analysis on the sampled data

# Example: Calculate the mean balance of the sampled population

mean_balance = sampled_data['balance'].mean() print(f"\nMean

balance of the sampled population: {mean_balance}")

# Example: Count occurrences of each job type in the sample
```

```
job_counts = sampled_data['job'].value_counts() print("\nJob
counts in the sampled population:") print(job_counts)
# Step 6: Save the sampled data to a new CSV file (optional)
sampled_data.to_csv('sampled_data.csv', index=False) print("\nSampled
data has been saved to 'sampled_data.csv'.")
```

Output:

```
First 5 rows of the dataset:
   age        job  marital  education  ... pdays  previous poutcome deposit 0
59     admin.  married  secondary  ...    -1       0  unknown     yes
1   56     admin.  married  secondary  ...    -1       0  unknown     yes
2   41  technician  married  secondary  ...    -1       0  unknown     yes
3   55    services  married  secondary  ...    -1       0  unknown     yes
4   54     admin.  married   tertiary  ...    -1       0  unknown     yes

[5 rows x 17 columns]

Sampled Data:
     age        job   marital  education  ... pdays  previous poutcome deposit 8422
31      admin.    single  secondary  ...    -1       0  unknown      no
9226   43  blue-collar   married  secondary  ...    -1       0  unknown      no
8162   26  blue-collar    single  secondary  ...    -1       0  unknown      no
4337   39     services    single  secondary  ...   185       1  success     yes 2438
40   management   married   tertiary  ...    98       6  success     yes
...   ...        ...      ...       ... ...  ...      ...     ...     ...
6586   50   technician  divorced  secondary  ...    -1       0  unknown      no
9168   55   technician    married    unknown  ...    -1       0  unknown      no 5322   26  blue-
collar    single  secondary  ...    -1       0  unknown      no
5152   32   management    single   tertiary  ...   375       5  failure     yes
```

1167 31 unemployed single secondary ... -1 0 unknown yes

[100 rows x 17 columns]

Mean balance of the sampled population: 2038.26

Job counts in the sampled population:

job management

21

blue-collar 19

technician 13

admin. 11 services

9

entrepreneur 6 unemployed

6

student 4 retired

4

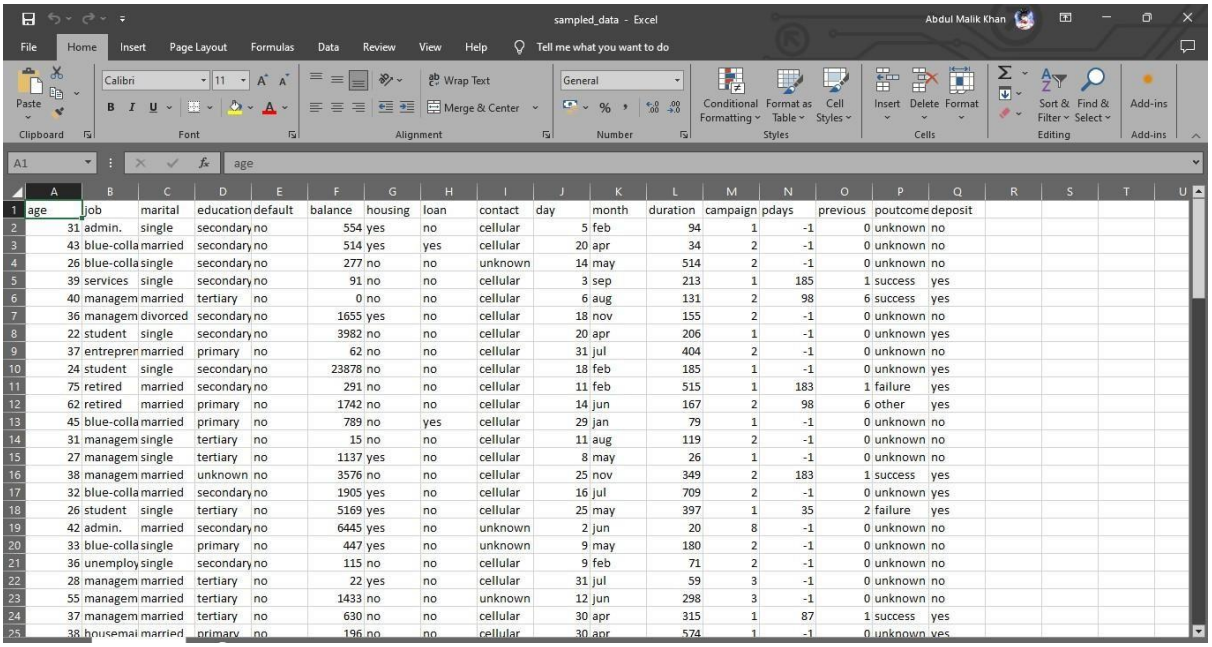
self-employed 4 housemaid

3

Name: count, dtype: int64

Sampled data has been saved to 'sampled_data.csv'.

Below Down we See the Sampled_data.csv File Saved after using the Dataset file in the Code to perform the sampling techniques are employed.



age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	deposit
31	admin.	single	secondary	no	554	yes	no	cellular	5	feb	94	1	-1	0	unknown	no
43	blue-colla	married	secondary	no	514	yes	yes	cellular	20	apr	34	2	-1	0	unknown	no
26	blue-colla	single	secondary	no	277	no	no	unknown	14	may	514	2	-1	0	unknown	no
39	services	single	secondary	no	91	no	no	cellular	3	sep	213	1	185	1	success	yes
40	managem	married	tertiary	no	0	no	no	cellular	6	aug	131	2	98	6	success	yes
36	managem	divorced	secondary	no	1655	yes	no	cellular	18	nov	155	2	-1	0	unknown	no
22	student	single	secondary	no	3982	no	no	cellular	20	apr	206	1	-1	0	unknown	yes
37	entrepren	married	primary	no	62	no	no	cellular	31	jul	404	2	-1	0	unknown	no
24	student	single	secondary	no	23878	no	no	cellular	18	feb	185	1	-1	0	unknown	yes
75	retired	married	secondary	no	291	no	no	cellular	11	feb	515	1	183	1	failure	yes
62	retired	married	primary	no	1742	no	no	cellular	14	jun	167	2	98	6	other	yes
45	blue-colla	married	primary	no	789	no	yes	cellular	29	jan	79	1	-1	0	unknown	no
31	managem	single	tertiary	no	15	no	no	cellular	11	aug	119	2	-1	0	unknown	no
27	managem	single	tertiary	no	1137	yes	no	cellular	8	may	26	1	-1	0	unknown	no
38	managem	married	unknown	no	3576	no	no	cellular	25	nov	349	2	183	1	success	yes
32	blue-colla	married	secondary	no	1905	yes	no	cellular	16	jul	709	2	-1	0	unknown	yes
26	student	single	tertiary	no	5169	yes	no	cellular	25	may	397	1	35	2	failure	yes
42	admin.	married	secondary	no	6445	yes	no	unknown	2	jun	20	8	-1	0	unknown	no
33	blue-colla	single	primary	no	447	yes	no	unknown	9	may	180	2	-1	0	unknown	no
36	unemploy	single	secondary	no	115	no	no	cellular	9	feb	71	2	-1	0	unknown	no
28	managem	married	tertiary	no	22	yes	no	cellular	31	jul	59	3	-1	0	unknown	no
55	managem	married	tertiary	no	1433	no	no	unknown	12	jun	298	3	-1	0	unknown	no
37	managem	married	tertiary	no	630	no	no	cellular	30	apr	315	1	87	1	success	yes
38	housemai	married	primary	no	196	no	no	cellular	30	apr	574	1	-1	0	unknown	yes

7(B) Use the Central Limit Theorem to analyze the sampling distribution of a sample mean and estimate population parameters

Code:

```
import pandas as pd import
numpy as np import
matplotlib.pyplot as plt import
seaborn as sns
# Load the data from the CSV file data
= pd.read_csv('bank.csv')
# Step 1: Calculate population parameters
population_mean = data['balance'].mean()
population_std = data['balance'].std() print(f"Population
Mean: {population_mean}") print(f"Population Standard
Deviation: {population_std}")
# Step 2: Define parameters for sampling
sample_size = 30  # Sample size num_samples =
1000  # Number of samples to draw # Step 3: Draw
samples and calculate sample means
sample_means = [] for _ in range(num_samples):
    sample = data['balance'].sample(n=sample_size, random_state=None)
sample_means.append(sample.mean())
# Step 4: Calculate mean and standard deviation of sample means
sampling_mean = np.mean(sample_means) sampling_std =
np.std(sample_means) print(f"\nMean of Sample Means:
{sampling_mean}") print(f"Standard Deviation of Sample Means:
{sampling_std}")
# Step 5: Central Limit Theorem Analysis
# According to CLT, the sampling distribution of the sample mean should have:
# Mean = population mean
```

```
# Standard deviation = population standard deviation / sqrt(sample size) clt_mean
= population_mean
clt_std = population_std / np.sqrt(sample_size)
print(f"\nCLT Estimated Mean: {clt_mean}") print(f"CLT
Estimated Standard Deviation: {clt_std}")
# Step 6: Visualize the sampling distribution of the sample means plt.figure(figsize=(12,
6))
sns.histplot(sample_means, bins=30, kde=True, stat="density", color='blue', label='Sample
Means')
# Add vertical lines for the population mean, sample mean, and CLT mean
plt.axvline(population_mean, color='red', linestyle='dashed', linewidth=2, label='Population
Mean')
plt.axvline(sampling_mean, color='green', linestyle='dashed', linewidth=2, label='Mean of
Sample Means')
plt.axvline(clt_mean, color='orange', linestyle='dashed', linewidth=2, label='CLT Estimated
Mean')
# Plot CLT standard deviation (one standard deviation interval around the mean)
plt.axvline(clt_mean - clt_std, color='purple', linestyle='dashed', linewidth=2, label='CLT
Mean - 1 SD')
plt.axvline(clt_mean + clt_std, color='purple', linestyle='dashed', linewidth=2, label='CLT
Mean + 1 SD') plt.title('Sampling Distribution of Sample Means (Central Limit
Theorem)') plt.xlabel('Sample Mean Balance')
plt.ylabel('Density')
plt.legend() plt.show()
```

Output:

Population Mean: 1528.5385235620856

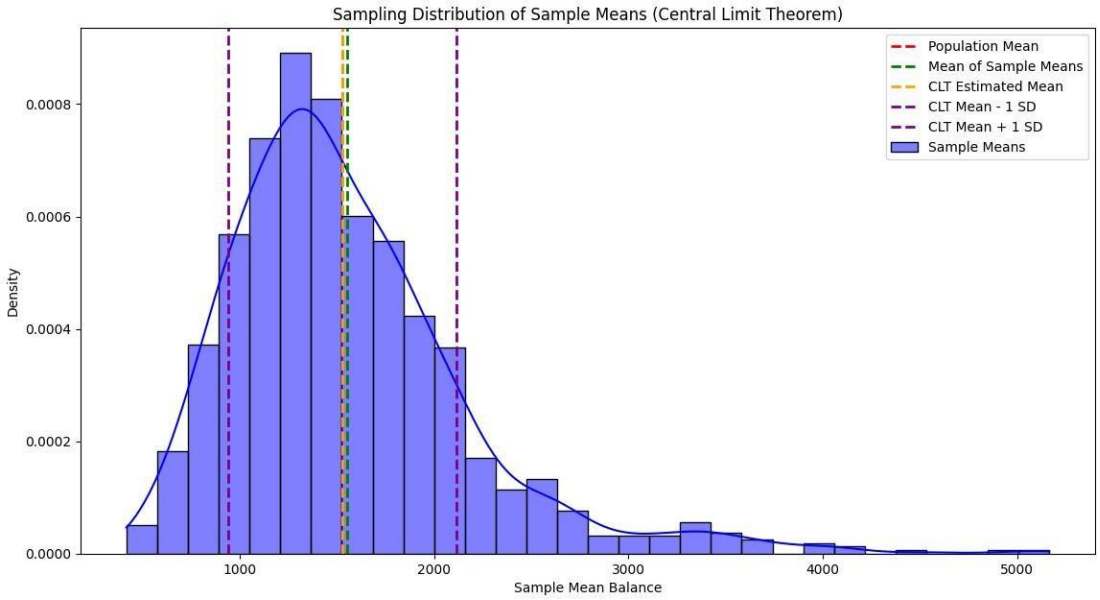
Population Standard Deviation: 3225.413325946151

Mean of Sample Means: 1550.0666333333334

Standard Deviation of Sample Means: 622.7231342409625

CLT Estimated Mean: 1528.5385235620856

CLT Estimated Standard Deviation: 588.8772119661566



Applied Business Analytics # Practical No: 8

Name		Roll No:	46
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Confidence Interval Estimation:		

Practical No: 8 Confidence Interval Estimation:

8(A) Calculate confidence intervals for population means or proportions using sample data and interpret the results in a business context.

Code:

```
# Import necessary libraries import numpy as np import pandas as pd from
scipy import stats import matplotlib.pyplot as plt # Step 1: Load the dataset
data = pd.read_csv('students_exam_scores.csv')  # Adjust file path if
necessary
# Step 2: Display the first few rows and column names
print(data.head()) print("Column names:",
data.columns)
# Step 3: Extract Math scores (adjust column name if necessary) math_scores =
data['MathScore']  # Change this if needed based on printed column names
# Step 4: Calculate sample mean and standard deviation sample_mean =
np.mean(math_scores) sample_std_dev = np.std(math_scores, ddof=1)  #
Sample standard deviation sample_size = len(math_scores)
# Step 5: Calculate confidence interval for the population mean
confidence_level = 0.95 alpha = 1 - confidence_level t_value =
stats.t.ppf(1 - alpha / 2, df=sample_size - 1)  # Two-tailed t-value
margin_of_error = t_value * (sample_std_dev / np.sqrt(sample_size))
```

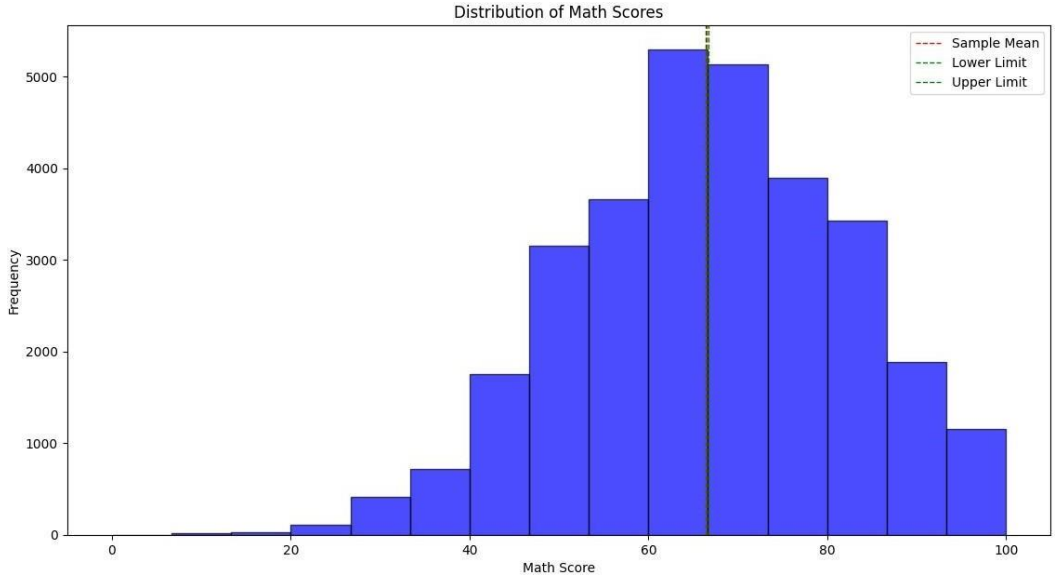
```
lower_limit = sample_mean - margin_of_error upper_limit =

sample_mean + margin_of_error

# Print the sample mean and confidence interval for the population mean print(f"Sample

Mean Math Score: {sample_mean:.2f}")

print(f"Confidence Interval for Population Mean Math Score: ({lower_limit:.2f},
{upper_limit:.2f})")

# Optional: Visualize the Math scores data plt.hist(math_scores, bins=15,

alpha=0.7, color='blue', edgecolor='black') plt.axvline(sample_mean,

color='red', linestyle='dashed', linewidth=1) plt.axvline(lower_limit,

color='green', linestyle='dashed', linewidth=1) plt.axvline(upper_limit,

color='green', linestyle='dashed', linewidth=1) plt.title('Distribution of

Math Scores') plt.xlabel('Math Score') plt.ylabel('Frequency')

plt.legend(['Sample Mean', 'Lower Limit', 'Upper Limit']) plt.show()

# Business Context for Population Mean: print("\nBusiness

Context Interpretation (Population Mean):")

print(f"We are {confidence_level*100}% confident that the true average math score for the
entire population of students lies between {lower_limit:.2f} and {upper_limit:.2f}.")

# Step 6: Calculate confidence interval for population proportion

# Assume we want to calculate the proportion of students who scored above a certain
threshold, e.g., 70 threshold = 70

num_successes = np.sum(math_scores > threshold)  # Count how many students scored
above the threshold num_trials = sample_size  # Total number of students (sample size)

sample_proportion = num_successes / num_trials  # Proportion of successes

# Step 7: Calculate confidence interval for population proportion z_value =

stats.norm.ppf(1 - alpha / 2)  # Z-score for a 95% confidence interval

margin_of_error_proportion = z_value * np.sqrt((sample_proportion * (1 -
sample_proportion)) / num_trials) lower_limit_proportion =
sample_proportion - margin_of_error_proportion upper_limit_proportion =
sample_proportion + margin_of_error_proportion

# Print results for population proportion print(f"Sample Proportion of Students Scoring

Above {threshold}: {sample_proportion:.2f}")
```

```
print(f"Confidence Interval for Proportion of Students Scoring Above {threshold}: ({lower_limit_proportion:.2f}, {upper_limit_proportion:.2f})")

# Business Context for Population Proportion: print("\nBusiness

Context Interpretation (Population Proportion):")

print(f"We are {confidence_level*100}% confident that the true proportion of students who scored above {threshold} lies between {lower_limit_proportion:.2f} and {upper_limit_proportion:.2f}.")
```

Output:

```
   Unnamed: 0  Gender EthnicGroup  ... MathScore ReadingScore WritingScore

0           0  female        NaN  ...        71          71          74

1           1  female    group C  ...        69          90          88 2           2  female

            group B  ...        87          93          91

3           3    male    group A  ...        45          56          42

4           4    male    group C  ...        76          78          75

[5 rows x 15 columns]

Column names: Index(['Unnamed: 0', 'Gender', 'EthnicGroup', 'ParentEduc', 'LunchType',

       'TestPrep', 'ParentMaritalStatus', 'PracticeSport', 'IsFirstChild',

       'NrSiblings', 'TransportMeans', 'WklyStudyHours', 'MathScore',

       'ReadingScore', 'WritingScore'],

      dtype='object')

Sample Mean Math Score: 66.56

Confidence Interval for Population Mean Math Score: (66.39, 66.73)

Business Context Interpretation (Population Mean):

We are 95.0% confident that the true average math score for the entire population of students lies between 66.39 and 66.73.

Sample Proportion of Students Scoring Above 70: 0.41

Confidence Interval for Proportion of Students Scoring Above 70: (0.41, 0.42)

Business Context Interpretation (Population Proportion):
```

We are 95.0% confident that the true proportion of students who scored above 70 lies between 0.41 and 0.42.



8(B) Apply bootstrapping techniques to estimate confidence intervals for non-parametric statistics.

Code:

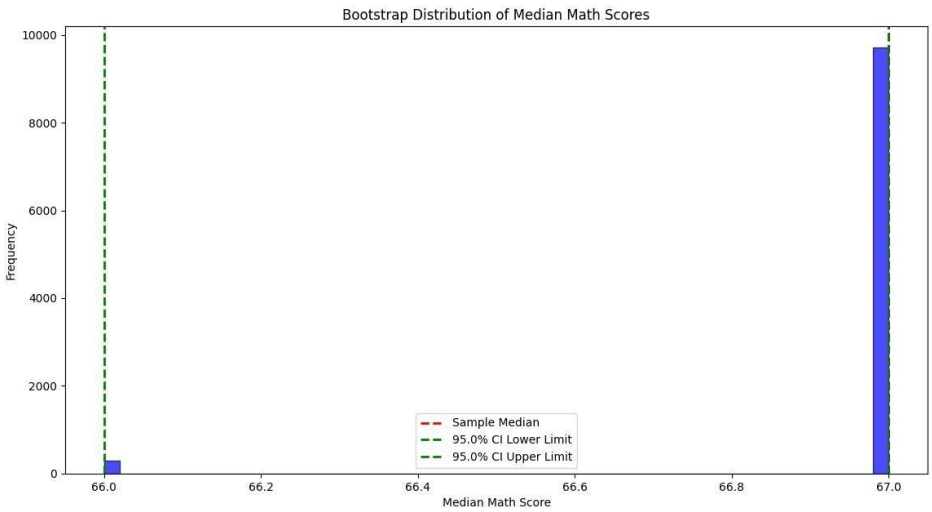
```
import numpy as np import

pandas as pd import

matplotlib.pyplot as plt

# Step 1: Load your data (Example: Exam Scores)

data = pd.read_csv('D:/Sem 6 Practicals/ABA Practicals/P8/students_exam_scores.csv') # Adjust file path if necessary

# Display the first few rows and column names

print(data.head()) print("Column names:",

data.columns)
```

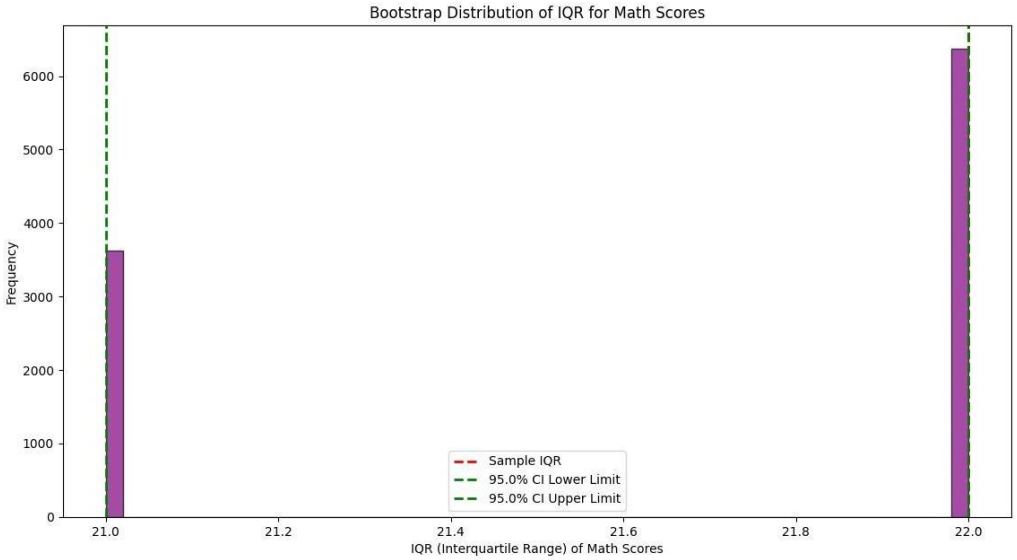
```
# Step 2: Extract the relevant column for bootstrapping (e.g., MathScores) math_scores
= data['MathScore']  # Adjust the column name if necessary
# Step 3: Define the bootstrapping function def
bootstrap(data, num_iterations, statistic_func):
# Create an array to store the bootstrap statistics
bootstrap_statistics = np.zeros(num_iterations)
   # Generate bootstrap samples and compute the statistic of interest
for i in range(num_iterations):
      sample = np.random.choice(data, size=len(data), replace=True)  # Resample with
replacement
      bootstrap_statistics[i] = statistic_func(sample)  # Compute the statistic (e.g., median)


   return bootstrap_statistics
# Step 4: Apply bootstrapping to estimate confidence intervals for the median
num_iterations = 10000  # Number of bootstrap samples
bootstrap_statistics = bootstrap(math_scores, num_iterations, np.median)  # Using median
as the statistic
# Step 5: Calculate the confidence intervals from the bootstrap statistics
confidence_level = 0.95 lower_percentile = (1 - confidence_level) / 2 *
100 upper_percentile = (1 + confidence_level) / 2 * 100 lower_limit =
np.percentile(bootstrap_statistics, lower_percentile) upper_limit =
np.percentile(bootstrap_statistics, upper_percentile)
# Step 6: Print the results
print(f"Bootstrap Confidence Interval for Median Math Score: ({lower_limit:.2f},
{upper_limit:.2f})")
# Optional: Visualize the bootstrap distribution plt.hist(bootstrap_statistics,
bins=50, alpha=0.7, color='blue', edgecolor='black')
plt.axvline(np.median(math_scores), color='red', linestyle='dashed', linewidth=2,
label='Sample Median')
plt.axvline(lower_limit, color='green', linestyle='dashed', linewidth=2,
label=f'{confidence_level*100}% CI Lower Limit')
```

```
plt.axvline(upper_limit, color='green', linestyle='dashed', linewidth=2,
label=f'{confidence_level*100}% CI Upper Limit') plt.title('Bootstrap
Distribution of Median Math Scores') plt.xlabel('Median Math Score')
plt.ylabel('Frequency') plt.legend() plt.show()
# Optional: Apply bootstrapping for other non-parametric statistics like the interquartile
range (IQR) def bootstrap_iqr(data, num_iterations):    iqr_statistics =
np.zeros(num_iterations)     for i in range(num_iterations):
    sample = np.random.choice(data, size=len(data), replace=True)
q25, q75 = np.percentile(sample, [25, 75])        iqr_statistics[i] =
q75 - q25     return iqr_statistics

# Step 7: Apply bootstrapping to estimate confidence intervals for the IQR
bootstrap_iqr_values = bootstrap_iqr(math_scores, num_iterations)
# Calculate the 95% confidence interval for IQR lower_iqr_percentile =
np.percentile(bootstrap_iqr_values, lower_percentile) upper_iqr_percentile =
np.percentile(bootstrap_iqr_values, upper_percentile)
# Print results for IQR
print(f"Bootstrap Confidence Interval for IQR of Math Scores: ({lower_iqr_percentile:.2f},
{upper_iqr_percentile:.2f})")
# Optional: Visualize the bootstrap distribution for IQR plt.hist(bootstrap_iqr_values,
bins=50, alpha=0.7, color='purple', edgecolor='black')
plt.axvline(np.percentile(math_scores, 75) - np.percentile(math_scores, 25), color='red',
linestyle='dashed', linewidth=2, label='Sample IQR')
plt.axvline(lower_iqr_percentile, color='green', linestyle='dashed', linewidth=2,
label=f'{confidence_level*100}% CI Lower Limit')
plt.axvline(upper_iqr_percentile, color='green', linestyle='dashed', linewidth=2,
label=f'{confidence_level*100}% CI Upper Limit') plt.title('Bootstrap
Distribution of IQR for Math Scores') plt.xlabel('IQR (Interquartile Range) of
Math Scores') plt.ylabel('Frequency') plt.legend() plt.show()
```

Output:

```
  Unnamed: 0  Gender EthnicGroup  ... MathScore ReadingScore WritingScore
0           0  female        NaN  ...        71          71          74
1           1  female    group C  ...        69          90          88 2           2  female
            group B  ...        87          93          91
3           3    male    group A  ...        45          56          42
4           4    male    group C  ...        76          78          75

[5 rows x 15 columns]
Column names: Index(['Unnamed: 0', 'Gender', 'EthnicGroup', 'ParentEduc', 'LunchType',
       'TestPrep', 'ParentMaritalStatus', 'PracticeSport', 'IsFirstChild',
       'NrSiblings', 'TransportMeans', 'WklyStudyHours', 'MathScore',
       'ReadingScore', 'WritingScore'],
dtype='object')
Bootstrap Confidence Interval for Median Math Score: (66.00, 67.00)
Bootstrap Confidence Interval for IQR of Math Scores: (21.00, 22.00)
```




Bootstrap Distribution of IQR for Math Scores
Frequency
0
1000
2000
3000
4000
5000
6000
21.0
21.2
21.4
21.6
21.8
22.0
IQR (Interquartile Range) of Math Scores
Sample IQR
95.0% CI Lower Limit
95.0% CI Upper Limit

Applied Business Analytics # Practical No: 9

Name		**Roll No:**	
Class	T.Y.B.Sc. Data Science	**Seat Number:**	
Subject / Topic	Applied Business Analytics		
Topic	Hypothesis Testing:		

Practical No: 9 Hypothesis Testing:

9(A) Formulate null and alternative hypotheses related to a business problem, conduct a hypothesis test using appropriate statistical tests, and interpret the results.

Code:

```
import pandas as pd import

numpy as np from scipy

import stats import

matplotlib.pyplot as plt

# Load the Yulu Bike Sharing dataset

# Make sure to replace 'yulu_data.csv' with the actual path to your dataset

yulu_data = pd.read_csv('D:/Sem 6 Practicals/ABA
Practicals/P9/yulu_bike_sharing_dataset.csv')

# Display the first few rows of the dataset to understand its structure

print("First few rows of the dataset:") print(yulu_data.head())

# Check for any missing values in the relevant columns

print("\nMissing values in 'count' and 'workingday' columns:")

print(yulu_data[['count', 'workingday']].isnull().sum()) # Separate data

into working days and non-working days working_days =

yulu_data[yulu_data['workingday'] == 1]['count'] non_working_days =

yulu_data[yulu_data['workingday'] == 0]['count']

# Formulate the null and alternative hypothes6e8s:
```

Null Hypothesis (H_0): There is no significant difference in bike rentals between working and non-working days.

Alternative Hypothesis (H_1): There is a significant difference in bike rentals between

working and non-working days. # Conducting a two-sample t-test t_statistic, p_value

= stats.ttest_ind(working_days, non_working_days)

Output the results of the hypothesis test

print(f"\nT-statistic: {t_statistic:.4f}")

print(f"P-value: {p_value:.4f}") # Set

significance level alpha = 0.05

Decision rule based on p-value if

p_value <= alpha:

print("Reject the null hypothesis: There is a significant difference in bike rentals between working and non-working days.")

else:

print("Fail to reject the null hypothesis: There is no significant difference in bike rentals between working and non-working days.")

Optional: Visualizing the distributions of bike rentals on working and non-working days

plt.figure(figsize=(10, 6)) plt.hist(working_days, alpha=0.5, label='Working Days', bins=15,

color='blue') plt.hist(non_working_days, alpha=0.5, label='Non-Working Days', bins=15,

color='orange')

plt.title('Distribution of Bike Rentals')

plt.xlabel('Number of Rentals')

plt.ylabel('Frequency') plt.legend()

plt.grid()

plt.show()

Output:

First few rows of the dataset:

	datetime	season	holiday	...	casual	registered	count
0	2011-01-01 00:00:00	1	0	...	3	13	16
1	2011-01-01 01:00:00	1	0	...	8	6932	40

2 2011-01-01 02:00:00 1 0 ... 5 27 32

3 2011-01-01 03:00:00 1 0 ... 3 10 13

4 2011-01-01 04:00:00 1 0 ... 0 1 1

[5 rows x 12 columns]

Missing values in 'count' and 'workingday' columns:

count 0

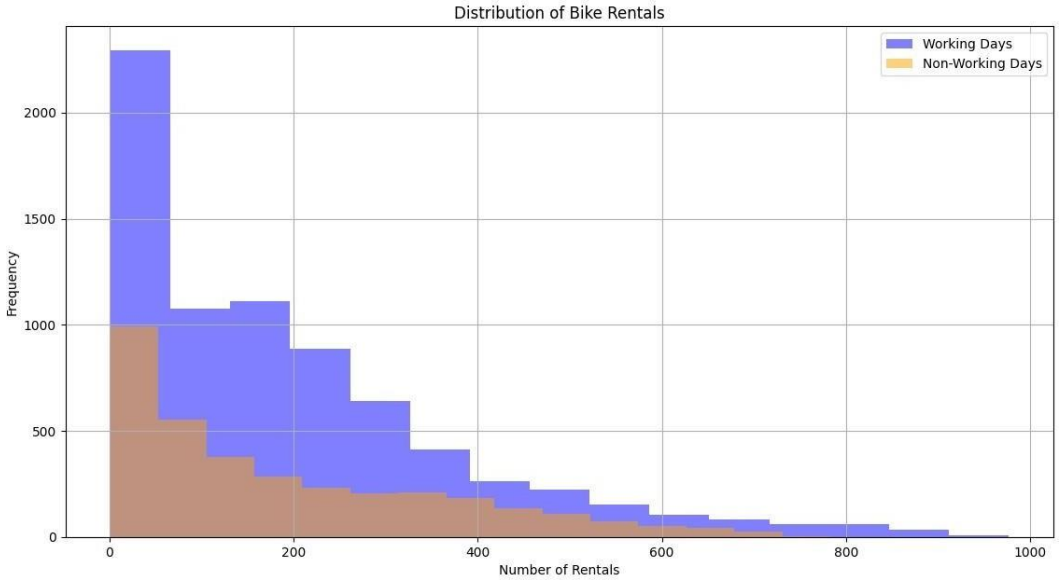
workingday 0 dtype:

int64

T-statistic: 1.2096

P-value: 0.2264

Fail to reject the null hypothesis: There is no significant difference in bike rentals between working and non-working days.



9(B) Perform A/B testing on a website or marketing campaign to evaluate the effectiveness of different strategies and make datadriven decisions.

Code:

```
import pandas as pd import numpy as np from statsmodels.stats.proportion
import proportions_ztest  # Correct import import matplotlib.pyplot as plt
# Load the dataset data = pd.read_csv('D:/Sem 6 Practicals/ABA
Practicals/P9/marketing_AB.csv')
# Display dataset overview print("Dataset
Overview:") print(data.info()) print("\nFirst
few rows of the dataset:") print(data.head())
# Check for null values print("\nMissing
values in relevant columns:") missing_values
= data.isnull().sum() print(missing_values)
# Ensure dataset integrity (drop rows with missing values if any)
if missing_values.sum() > 0:     data.dropna(inplace=True)
print("\nDropped rows with missing values.")

# Check unique values in the 'test group' column print("\nUnique
values in the 'test group' column:") print(data['test
group'].unique())
# Calculate conversion rates for 'ad' and 'psa' groups
conversion_rates = data.groupby('test group')['converted'].mean()
print("\nConversion Rates:") print(f"Conversion Rate for 'ad' Group:
{conversion_rates['ad']:.2%}") print(f"Conversion Rate for 'psa' Group:
{conversion_rates['psa']:.2%}")
# Prepare data for z-test count = np.array([
data[data['test group'] == 'ad']['converted'].sum(),
data[data['test group'] == 'psa']['converted'].sum()
])
nobs = np.array([     data[data['test group']
== 'ad'].shape[0],     data[data['test group']
== 'psa'].shape[0]
])
```

```
# Conduct z-test for proportions z_stat, p_value
= proportions_ztest(count, nobs)
# Output z-test results print("\nZ-Test
Results:") print(f"Z-statistic:
{z_stat:.4f}") print(f"P-value:
{p_value:.4f}")
# Hypothesis testing alpha =
0.05  # significance level if
p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference between 'ad' and 'psa'
groups.")     print("The marketing strategy significantly impacts conversion rates.") else:
    print("Fail to reject the null hypothesis: There is no significant difference between 'ad' and
'psa' groups.")     print("The marketing strategy does not significantly impact
conversion rates.")
# Optional: Visualize conversion rates
plt.figure(figsize=(8, 6)) plt.bar(['ad', 'psa'], [conversion_rates['ad'], conversion_rates['psa']],
color=['blue', 'orange']) plt.ylabel('Conversion Rate') plt.title('Conversion Rates for
Marketing Strategies') plt.ylim(0, 1) for i, rate in enumerate([conversion_rates['ad'],
conversion_rates['psa']]):
    plt.text(i, rate + 0.02, f"{rate:.2%}", ha='center', fontsize=12) plt.show()
# Insights and Recommendations
print("\nInsights and Recommendations:") if
p_value < alpha:
    print("The 'ad' or 'psa' strategy should be prioritized depending on the higher conversion
rate.")
    print("Further analysis on demographic segmentation may provide better targeted
strategies.") else:   print("Explore alternative strategies or optimize both groups for
improved effectiveness.")
```

Output:

```
Dataset Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 588101 entries, 0 to 588100 Data
columns (total 7 columns):
 #   Column         Non-Null Count   Dtype
--  ----           ----------       ----
0   Unnamed: 0     588101 non-null  int64
1   user id        588101 non-null  int64
2   test group     588101 non-null  object
3   converted      588101 non-null  bool
4   total ads      588101 non-null  int64
5   most ads day   588101 non-null  object  6   most ads hour  588101 non-null  int64  dtypes: bool(1),
    int64(4), object(2) memory usage: 27.5+ MB
None
First few rows of the dataset:
   Unnamed: 0  user id test group  ...  total ads  most ads day most ads hour
0           0  1069124        ad  ...        130       Monday           20
1           1  1119715        ad  ...         93      Tuesday           22
2           2  1144181        ad  ...         21      Tuesday           18
3           3  1435133        ad  ...        355      Tuesday           10
4           4  1015700        ad  ...        276       Friday           14
[5 rows x 7 columns]
Missing values in relevant columns: Unnamed:
0      0
user  id            0
test group       0
converted        0
total ads        0
most ads day     0 most
ads hour   0 dtype:
int64
```

Unique values in the 'test group' column:

['ad' 'psa']

Conversion Rates:

Conversion Rate for 'ad' Group: 2.55% Conversion

Rate for 'psa' Group: 1.79%

Z-Test Results:

Z-statistic: 7.3701

P-value: 0.0000

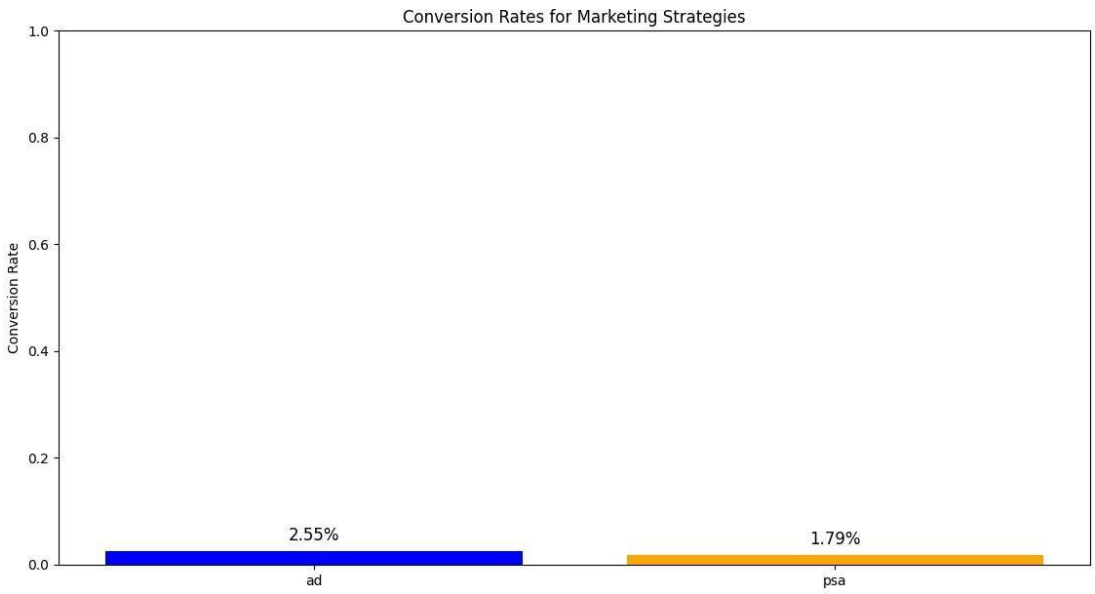
Reject the null hypothesis: There is a significant difference between 'ad' and 'psa' groups.

The marketing strategy significantly impacts conversion rates.

Insights and Recommendations:

The 'ad' or 'psa' strategy should be prioritized depending on the higher conversion rate.

Further analysis on demographic segmentation may provide better targeted strategies.



Applied Business Analytics # Practical No: 10

Name		Roll No:	
Class	T.Y.B.Sc. Data Science	Seat Number:	
Subject / Topic	Applied Business Analytics		
Topic	Regression Analysis and Time Series Analysis:		

Practical No: 10 Regression Analysis and Time Series Analysis:

10(A) Develop a regression model to predict future sales based on historical data, assess model performance, and interpret the significance of predictor variables.

Code:

```
#pip install pandas numpy matplotlib seaborn scikit-learn statsmodels
# Import necessary libraries import pandas as pd import
numpy as np import matplotlib.pyplot as plt import
seaborn as sns from sklearn.model_selection import
train_test_split from sklearn.linear_model import
LinearRegression from sklearn.metrics import
mean_squared_error, r2_score import statsmodels.api as
sm
# Step 1: Load the dataset
# Replace 'sales_data.csv' with your actual dataset path data =
pd.read_csv('D:/Sem 6 Practicals/ABA Practicals/P10/historical_sales.csv')
# Step 2: Data Preprocessing
# Convert 'Date' to datetime format and sort by date data['Date']
= pd.to_datetime(data['Date'], format='%Y%m%d')
data.sort_values('Date', inplace=True)
```

```
# Aggregate sales by date (if there are multiple entries per date) daily_sales
= data.groupby('Date')['Sold_Units'].sum().reset_index()
# Step 3: Exploratory Data Analysis (EDA) print("First few rows of the dataset:")
print(daily_sales.head()) plt.figure(figsize=(12, 6)) plt.plot(daily_sales['Date'],
daily_sales['Sold_Units'], label='Daily Sold Units', color='blue') plt.title('Daily Sold Units
Over Time') plt.xlabel('Date') plt.ylabel('Sold Units') plt.legend() plt.grid(True) plt.show()
# Step 4: Feature Engineering
# Create lagged features (previous day's sales)
daily_sales['Lagged_Sales'] = daily_sales['Sold_Units'].shift(1) #
Drop rows with missing values after creating lagged features
daily_sales.dropna(inplace=True)
# Prepare features (X) and target variable (y)
X = daily_sales[['Lagged_Sales']] y =
daily_sales['Sold_Units']
# Step 5: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Step 6: Initialize and fit the Linear Regression model
model = LinearRegression() model.fit(X_train,
y_train) # Step 7: Make predictions y_pred =
model.predict(X_test)
# Step 8: Evaluate model performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred) print(f'Mean
Squared Error (MSE): {mse:.2f}') print(f'R^2
Score: {r2:.2f}')
# Step 9: Analyze significance of predictors using OLS regression X_ols =
sm.add_constant(X_train)  # Add a constant term to the predictor
ols_model = sm.OLS(y_train, X_ols).fit() # Print the summary statistics
print("\nOLS Regression Summary:\n") print(ols_model.summary())
```

```
# Step 10: Visualize predictions vs actual sales plt.figure(figsize=(12, 6))

plt.scatter(y_test, y_pred, color='blue', alpha=0.6, label='Predictions')

plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2, label='Ideal Fit')

plt.xlabel('Actual Sold Units') plt.ylabel('Predicted Sold Units')

plt.title('Actual vs Predicted Sold Units') plt.legend() plt.grid(True)

plt.show()

# Step 11: Predict future sales

# Use the most recent sales data to predict the next day's sales recent_sales

= daily_sales.iloc[-1]['Lagged_Sales']  # Use the lagged feature

future_sales = model.predict(pd.DataFrame({'Lagged_Sales': [recent_sales]}))[0]  # Match
feature name

print(f"Predicted sales for the next day: {future_sales:.2f} units")
```

Output:

```
First few rows of the dataset:

      Date  Sold_Units

0 2017-01-01          1

1 2017-01-02          3

2 2017-01-03          4

3 2017-01-04          9

4 2017-01-05          5

Mean Squared Error (MSE): 88.88

R^2 Score: -1.24

OLS Regression Summary:

                            OLS Regression Results

==============================================================================
===

Dep. Variable:            Sold_Units   R-squared:                       0.313

Model:                           OLS   Adj. R-squared:                  0.312
```

Method: Least Squares F-statistic: 280.1

Date: Fri, 24 Jan 2025 Prob (F-statistic): 4.46e-52

Time: 15:19:09 Log-Likelihood: -2212.4

No. Observations: 616 AIC: 4429.

Df Residuals: 614 BIC: 4438.

Df Model: 1

Covariance Type: nonrobust

==========

coef std err t P>|t| [0.025 0.975] --------

---------- const

3.9142 0.633 6.182 0.000 2.671 5.158

Lagged_Sales 0.7081 0.042 16.736 0.000 0.625 0.791

==========

Omnibus: 833.814 Durbin-Watson: 1.927

Prob(Omnibus): 0.000 Jarque-Bera (JB): 229240.638

Skew: 6.779 Prob(JB): 0.00

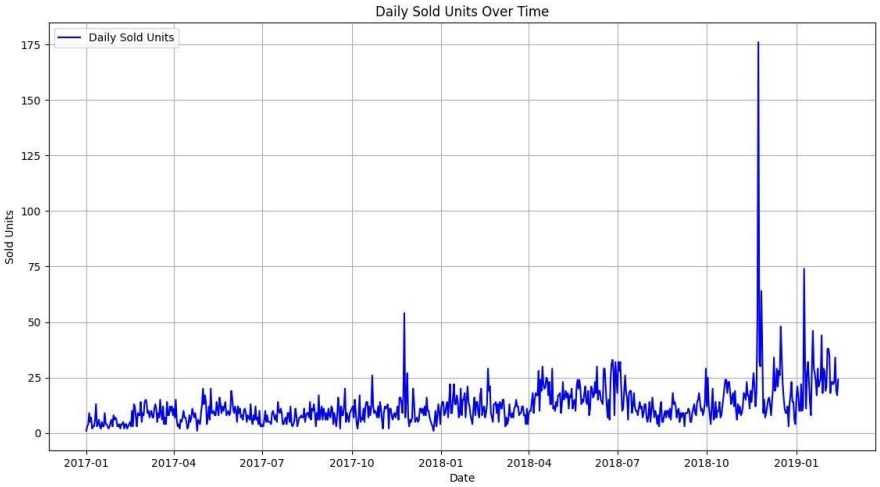
Kurtosis: 96.529 Cond. No. 26.8

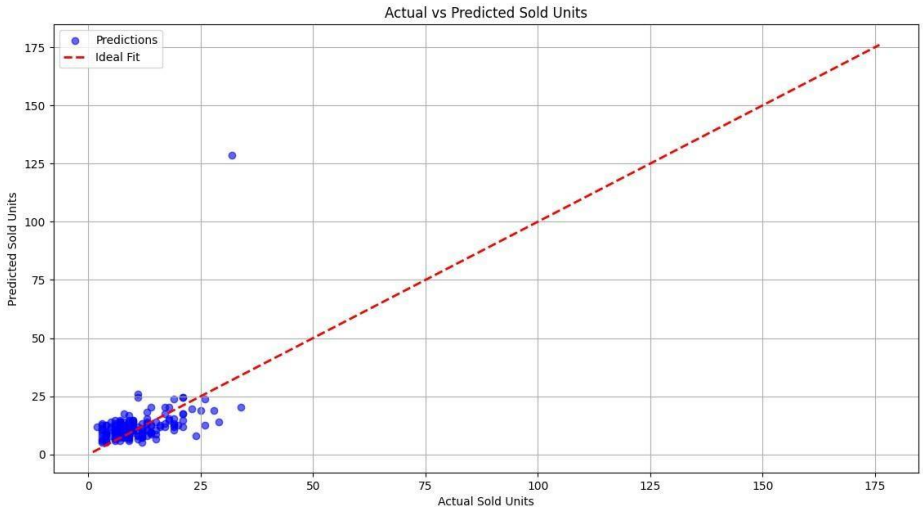
==========

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Predicted sales for the next day: 15.95 units





10(B) Apply time series analysis techniques (e.g., ARIMA, exponential smoothing) to forecast future demand for a product or service, and evaluate the accuracy of the forecasts.

Code:

Import necessary libraries import pandas as pd import numpy as np

import matplotlib.pyplot as plt from statsmodels.tsa.arima.model

import ARIMA from statsmodels.tsa.holtwinters import

ExponentialSmoothing from sklearn.metrics import

mean_squared_error, mean_absolute_error import warnings # Suppress

warnings warnings.filterwarnings('ignore') # Function to evaluate

forecasts def evaluate_forecast(test_data, forecast):

```
    mse = mean_squared_error(test_data, forecast)      rmse = np.sqrt(mse)      mae
= mean_absolute_error(test_data, forecast)     return mse, rmse, mae # Step 1:
Load the dataset data = pd.read_csv('D:/Sem 6 Practicals/ABA
Practicals/P10/historical_sales.csv')
# Step 2: Data Preprocessing data['Date'] =
pd.to_datetime(data['Date'], format='%Y%m%d')
data.sort_values('Date', inplace=True)
# Aggregate sales by date (if there are multiple entries per date) daily_sales
= data.groupby('Date')['Sold_Units'].sum().reset_index()
daily_sales.set_index('Date', inplace=True)

# Ensure the index has a frequency daily_sales =
daily_sales.asfreq('D', method='pad') # Step 3: Exploratory Data
Analysis (EDA) plt.figure(figsize=(12, 6))
plt.plot(daily_sales['Sold_Units'], label='Daily Sold Units', color='blue')
plt.title('Daily Sold Units Over Time') plt.xlabel('Date') plt.ylabel('Sold
Units') plt.legend() plt.grid(True) plt.show()
# Step 4: Train-Test Split train_size = int(len(daily_sales) * 0.8)
train, test = daily_sales.iloc[:train_size], daily_sales.iloc[train_size:]
# Step 5: Apply ARIMA Model arima_model = ARIMA(train['Sold_Units'], order=(5, 1, 0))
# Adjust (p, d, q) based on data arima_fit = arima_model.fit()  # fit() method handles
warnings internally print(arima_fit.summary()) # Forecast using ARIMA model
arima_forecast = arima_fit.forecast(steps=len(test))[0]  # Extract forecast values test =
test.copy()  # Avoid SettingWithCopyWarning test['ARIMA_Forecast'] = arima_forecast #
Step 6: Apply Exponential Smoothing Model exp_smoothing_model =
ExponentialSmoothing(
    train['Sold_Units'], trend='add', seasonal=None, initialization_method="estimated"
)
exp_fit = exp_smoothing_model.fit() exp_forecast
= exp_fit.forecast(steps=len(test))
```

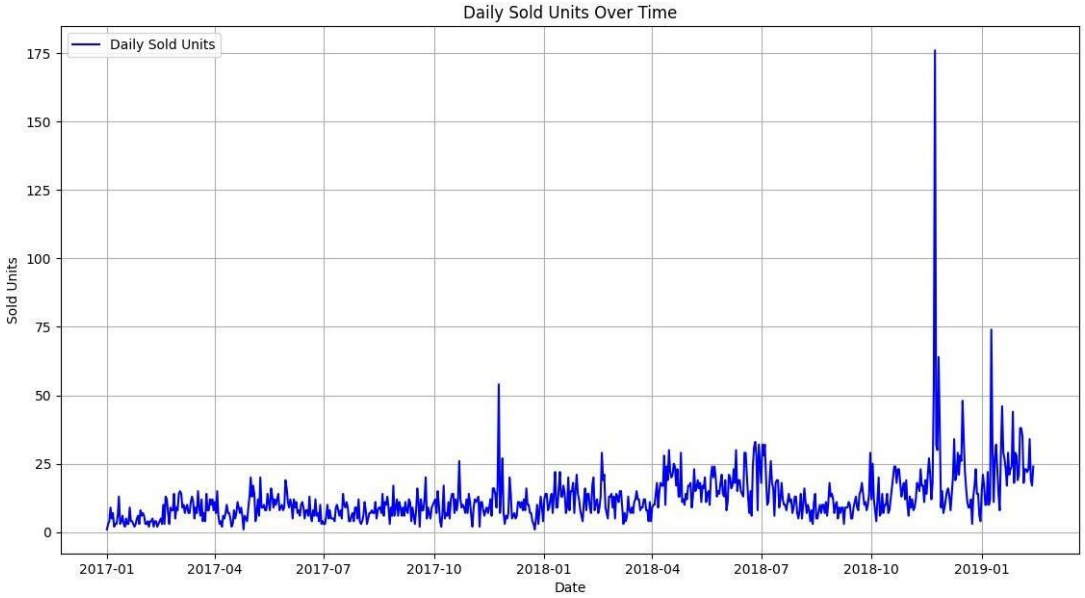
```
test['Exp_Smoothing_Forecast'] = exp_forecast #

Step 7: Evaluate Forecast Accuracy

arima_mse, arima_rmse, arima_mae = evaluate_forecast(test['Sold_Units'],
test['ARIMA_Forecast'])

exp_mse, exp_rmse, exp_mae = evaluate_forecast(test['Sold_Units'],

test['Exp_Smoothing_Forecast']) print(f'ARIMA - MSE: {arima_mse:.2f}, RMSE:

{arima_rmse:.2f}, MAE: {arima_mae:.2f}')

print(f'Exponential Smoothing - MSE: {exp_mse:.2f}, RMSE: {exp_rmse:.2f}, MAE:
{exp_mae:.2f}')

# Step 8: Visualize Forecasts vs Actual Sales plt.figure(figsize=(12, 6))

plt.plot(train.index, train['Sold_Units'], label='Train Data', color='blue')

plt.plot(test.index, test['Sold_Units'], label='Actual Sales', color='green')

plt.plot(test.index, test['ARIMA_Forecast'], label='ARIMA Forecast', color='red')

plt.plot(test.index, test['Exp_Smoothing_Forecast'], label='Exp Smoothing Forecast',

color='orange') plt.title('Forecast vs Actual Sales') plt.xlabel('Date') plt.ylabel('Sold

Units') plt.legend() plt.grid(True) plt.show()

# Step 9: Forecast Future Demand (Next 30 Days) future_steps

= 30 # ARIMA Future Forecast future_arima_forecast =

arima_fit.forecast(steps=future_steps)

# Exponential Smoothing Future Forecast future_exp_forecast

= exp_fit.forecast(steps=future_steps)

# Create a date range for future predictions

future_dates = pd.date_range(start=daily_sales.index[-1] + pd.Timedelta(days=1),
periods=future_steps, freq='D')

# Combine results into a DataFrame

future_forecast = pd.DataFrame({

    'Date': future_dates,

    'ARIMA_Forecast': future_arima_forecast,

    'Exp_Smoothing_Forecast': future_exp_forecast

}).set_index('Date') # Plot Future Forecasts plt.figure(figsize=(12, 6))

plt.plot(future_forecast['ARIMA_Forecast'], label='ARIMA Future Forecast', color='red')
```

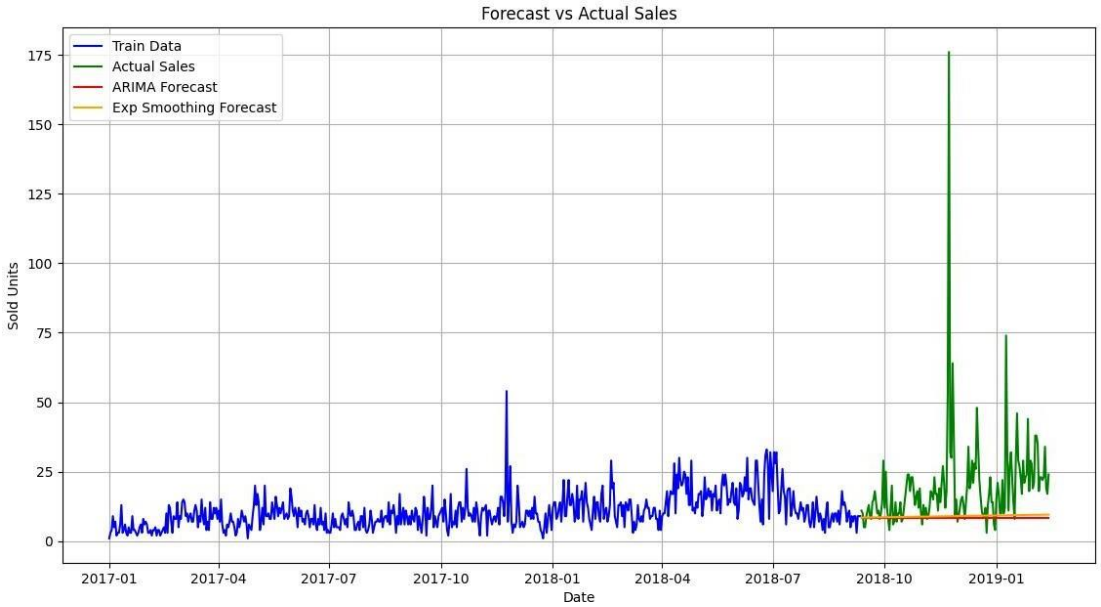
```
plt.plot(future_forecast['Exp_Smoothing_Forecast'], label='Exp Smoothing Future Forecast',
color='orange') plt.title('Future Demand Forecast (Next 30 Days)') plt.xlabel('Date')
plt.ylabel('Sold Units') plt.legend() plt.grid(True) plt.show()
```

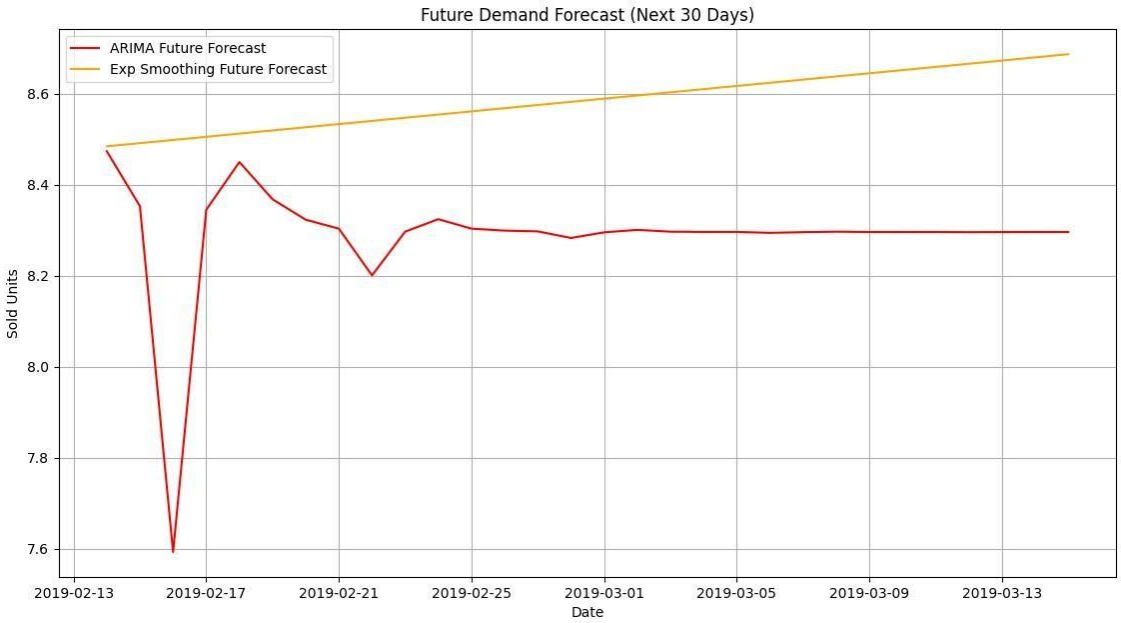
Output:

```
                               SARIMAX Results
==============================================================================
Dep. Variable:              Sold_Units   No. Observations:                  619
Model:                 ARIMA(5, 1, 0)   Log Likelihood               -1862.867
Date:                Fri, 24 Jan 2025   AIC                           3737.733
Time:                        15:30:09   BIC                           3764.292
Sample:                    01-01-2017   HQIC                          3748.058
                         - 09-11-2018
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975] ----------------------------------------
---------- ar.L1        -0.6575      0.023    -29.195      0.000      -0.702      -0.613 ar.L2        -0.5485      0.037    -
14.632      0.000      -0.622      -0.475 ar.L3        -0.3534      0.039     -9.032      0.000      -0.430      -0.277 ar.L4
-0.2657      0.032     -8.350      0.000      -0.328      -0.203 ar.L5        -0.1879      0.027     -6.854      0.000      -
0.242      -0.134 sigma2        24.2773      0.695     34.955      0.000      22.916      25.639
===================================================================================
=
Ljung-Box (L1) (Q):                   1.18   Jarque-Bera (JB):              1763.68
Prob(Q):                              0.28   Prob(JB):                         0.00
Heteroskedasticity (H):               2.62   Skew:                             0.96
Prob(H) (two-sided):                  0.00   Kurtosis:                        11.05
===================================================================================
=
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
ARIMA - MSE: 399.91, RMSE: 20.00, MAE: 11.55
```

Exponential Smoothing - MSE: 385.49, RMSE: 19.63, MAE: 11.09






Future Demand Forecast (Next 30 Days)
ARIMA Future Forecast
Exp Smoothing Future Forecast
Sold Units
8.6
8.4
8.2
8.0
7.8
7.6
2019-02-13
2019-02-17
2019-02-21
2019-02-25
2019-03-01
2019-03-05
2019-03-09
2019-03-13
Date